

HalleMalle

: 사이드 프로젝트 팀 구인을 위한 협업 플랫폼

소프트웨어학부 2110669 박민지
문헌정보학과 2510741 최다연



<https://hallemalle.vercel.app/>

목차

- 프로젝트 개요
- 문제 정의 / 대상 사용자
- 시장 및 유사 서비스 분석
- 기술 스택
- 기능 구현
- 설계 및 코드
- 어려웠던 점 및 해결 과정
- 협업/과정
- AI 활용 내역
- 결론 및 회고
- 참고자료

프로젝트 개요

프로젝트 개요

① 프로젝트명/슬로건

HalleMalle - “협업, 할래 말래?”

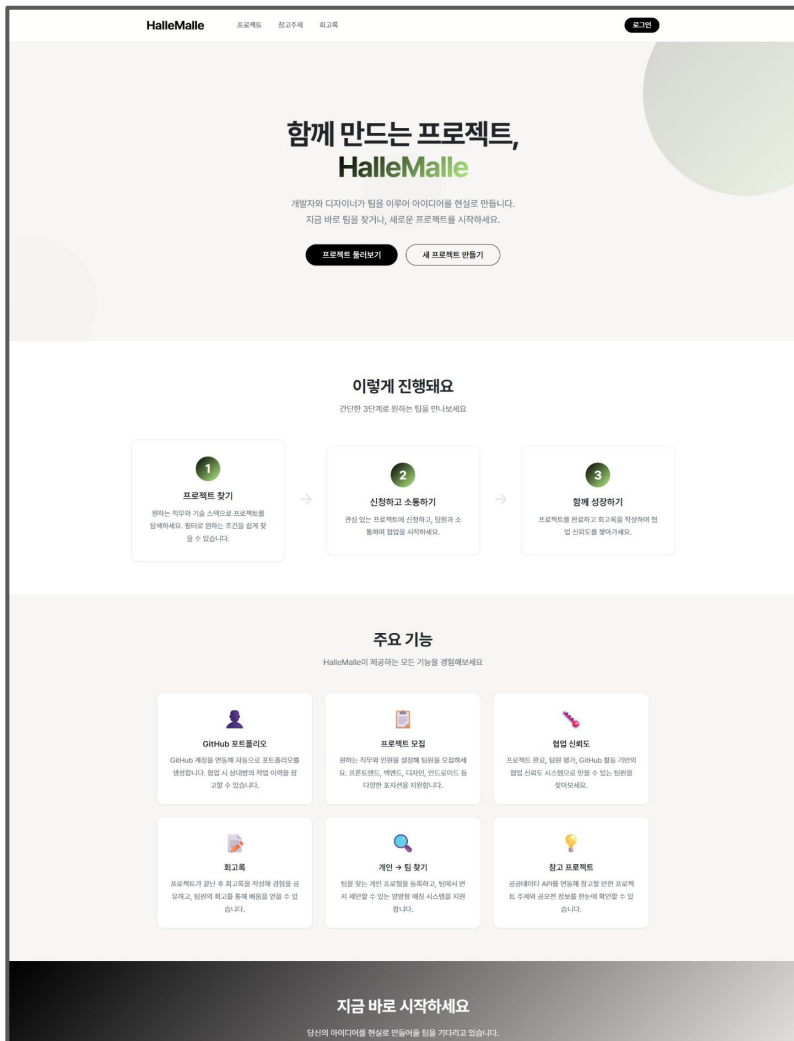
:사이드 프로젝트 팀 구인을 위한 협업 플랫폼

② 서비스 한 문단 설명

숙명여자대학교 교내 학우들이 협업 활동을 위해
게시글을 작성하고, 기획서와 포트폴리오를 주고
받으며 팀원을 모집하는 웹 서비스이다.

③ 핵심 가치 / 목표

- 신뢰성, 편의성, 효율성 증진
- 플랫폼을 통해 더욱 활발히, 함께 성장하기



프로젝트 개요

④ 배경

최근 대학생들은 교내 공모전, 팀 프로젝트, 스터디 등 다양한 협업 활동을 수행하며 실무 경험과 협업 역량을 축적한다. 팀원 모집 및 협업을 위해 에브리타임, 교내 학회/동아리 커뮤니티 기반의 디스코드, 지인 추천과 같은 신뢰 기반 네트워크를 사용한다. 그러나 이러한 모집 방식에는 다음과 같은 한계가 나타나며, 이 한계를 극복하고자 했다.



신뢰 기반 팀원 검증의 부재

포트폴리오를 주고 받아
신뢰성을 확보할 수
있다면?



협업 피드백의 단절

상대의 이전 협업 이력을 볼
수 있고, 협업 이후
피드백을 남길 수 있다면?



**정보의 분산 및 관리 비용
증가**

사전에 방향을 명확히
공유할 수 있고, 모집
절차가 효율적이라면?

문제 정의 / 대상 사용자

문제 정의 / 대상 사용자

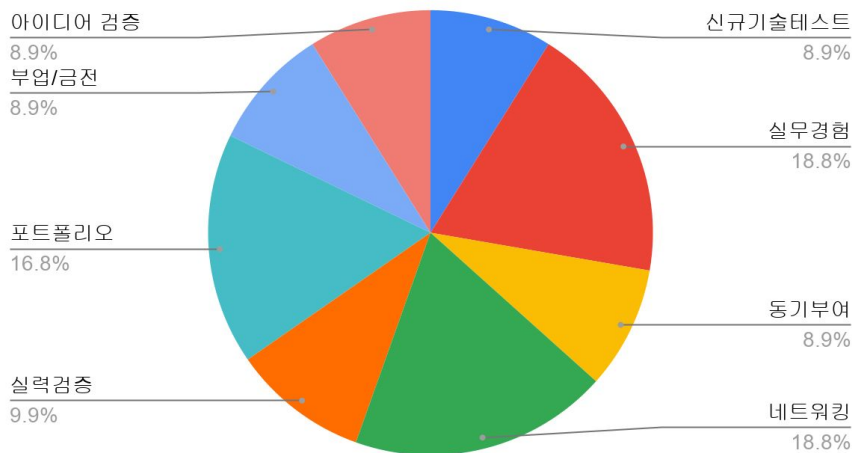
문제 정의 ①

**“사이드 프로젝트의 주 참여 동기는 네트워킹 ·
포트폴리오 · 실무경험이지만, 이를 범용적 · 적극적으로
자극하는 교내 문화가 부족함”**

커리어형 동기 54.4%

실무경험 · 네트워킹 · 포트폴리오가 주요 참여 동기

사이드 프로젝트의 참여 동기



출처: 렛플 인사이트 - 2024년 트렌드코리아 - 사이드프로젝트가 트렌드?!

문제 정의 / 대상 사용자

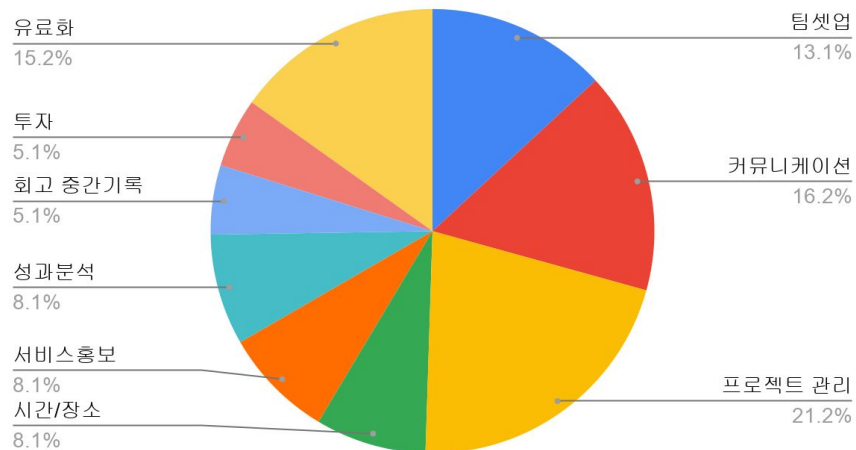
문제 정의 ②

“정해진 양식과 절차가 없어 모두 개인 역량에 의존하는
팀셋업, 프로젝트 관리, 의사소통 과정”

운영·협업 문제 50.5%

: 프로젝트 관리 21.2% · 커뮤니케이션 16.2% · 팀셋업 13.1%

사이드 프로젝트의 주요 문제 상황

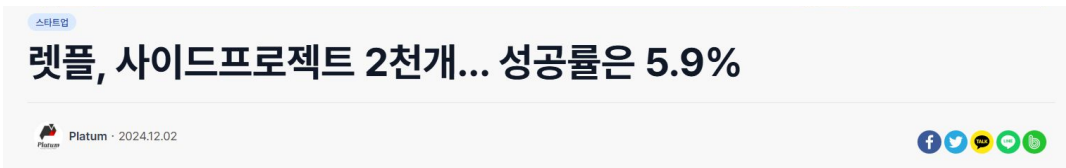


출처: 렛플 인사이트 - 2024년 트렌드코리아 - 사이드프로젝트가 트렌드?!

문제 정의 / 대상 사용자

문제 정의 ③

“사이드 프로젝트 팀을 모집하더라도,
실제 프로덕트 출시에 성공하기 어려움”



출처: Platum, 「렛플, 사이드프로젝트 2천개... 성공률은 5.9%」, 2024.12.02., 기사 원문

사이드 프로젝트 성공률 5.9%

: SF34가 운영하는 팀프로젝트 플랫폼 “렛플”에 등록된 사이드프로젝트 2000개 분석 결과, 실제 제품 출시까지 이어진 비율

팀원 이탈, 기획 및 프로젝트 관리 실패, 프로토타입 단계에서의 중단이 실패 원인으로 분석됨

문제 정의 / 대상 사용자

④ 유저 페르소나

숙명여대 소속 학생 중 팀 프로젝트·공모전·스터디를 위해
팀원을 모집하거나 참여하려는 사용자



**검증된 팀원을 찾고 싶은
프로젝트 리더**

- 주요 목표: 믿을 수 있는 개발 팀원을 빠르게 모집하고 싶다
- 핵심 니즈: 지원자의 실력, 사용 기술, 프로젝트 경험, 협업 태도를 사전에 확인하고 싶음



**좋은 팀에 들어가 포트폴리오를 쌓고 싶은
개발자 취준생**

- 주요 목표: 내 실력에 맞는 프로젝트를 찾고, GitHub 포트폴리오로 나를 어필하고 싶다
- 핵심 니즈: 팀을 쉽게 찾고, 별도 자기소개 없이 개발 경험을 자연스럽게 보여주고 싶음



**프로젝트에 참여하고 싶은
비개발자 타학과 협업자 (예정)**

- 주요 목표: 아이디어를 가진 상태에서 개발자와 팀을 이루고 싶다
- 핵심 니즈: GitHub가 없어도 자신의 기획·리서치 역량을 보여주고 싶음

시장 및 유사 서비스 분석

시장 및 유사 서비스 분석

① HOLA

“ 기술 스택 기반 스터디·사이드프로젝트
팀원 모집 플랫폼 ”

핵심 가치

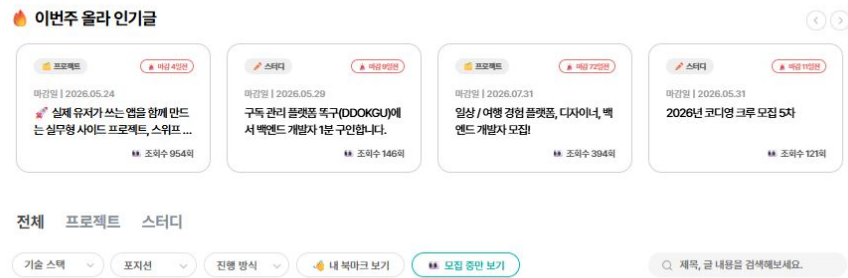
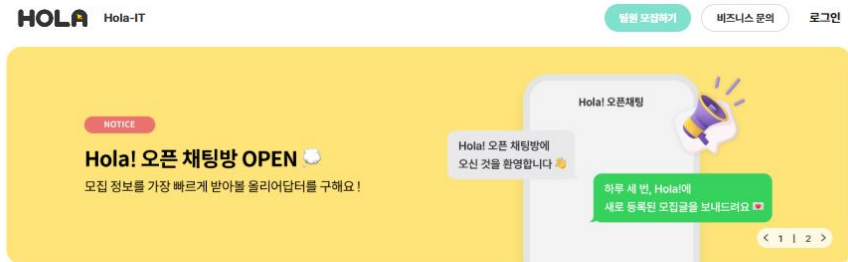
스택 기반 팀원 탐색 · 프로젝트/스터디 모집 · 공개형 모집 게시판

주요 기능

모집글 등록 · 기술스택 필터 · 포지션 필터 · 진행방식 필터 · 오픈채팅
연결

UX 특징

카드형 모집 리스트 · 인기글 상단 노출 · 모집중 필터 제공 · 북마크 기능
제공 · 검색/필터 병렬 배치



시장 및 유사 서비스 분석

② 렛플

“ 프로젝트·스터디를 커리어 경험으로
연결하는 플랫폼 ”

핵심 가치

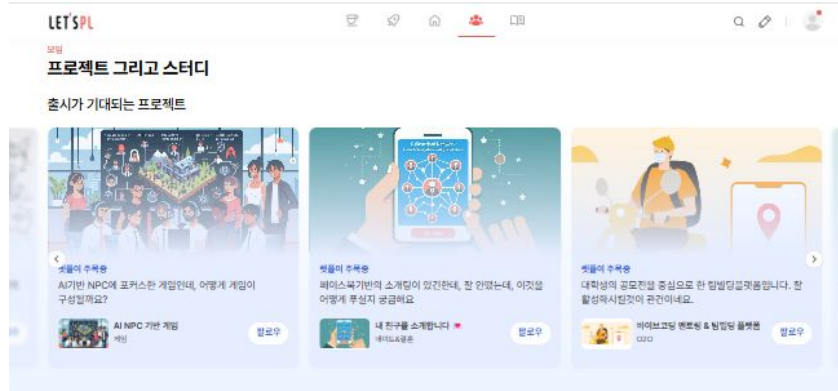
프로젝트/스터디 모집 · 팀원 연결 · 사이드프로젝트 이력화

주요 기능

모임 개설 · 프로젝트 지원 · 커피챗 신청 · 1:1/그룹채팅 · 서비스 홍보
· 이력서 관리

UX 특징

출시 예정 프로젝트 상단 노출 · 추천 모임 리스트 · 오늘의 모임
사이드바 · 카드형 광고 구조 · 광고형 공고 혼재



iaminsam님을 기다리는 모임

실무 중심 사이트 프로젝트

실선 수직화 및 개발 프로젝트

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

오늘의 모임

26.05.21 목요일

프로젝트/스터디 모집

실선 수직화 및 개발 프로젝트

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

04/04/04 ~ 09/09/09

시장 및 유사 서비스 분석

④ 할래말래의 차별점 (Why US?)

	기존 레퍼런스	할래말래
탐색 범위	전국·직군 혼합	교내 학우 풀
역량 검증	자기소개·링크 의존	GitHub 포트폴리오 검증
지원 운영	댓글·채팅 분산	신청관리 대시보드
협업 자료	링크 공유 중심	기획서·포트폴리오 교환

할래말래의 기회 = 교내 신뢰, 포트폴리오 검증, 신청관리의 결합

✓ 할래말래는 넓게 모집하지 않고 교내 협업을 증거로 연결한다.

주요 기능

주요 기능 설명

① 프로젝트 팀원 구인하기 및 신청하기 기능 (게시물 형태)

정해진 형식에 기입하여 게시물 형태로 팀원을 구인한다.
타 이용자들은 구인 리스트 페이지에서 읽기 및 신청이 가능하며, 작성자는 마이페이지에서 쓴 게시물들을 확인할 수 있다.

기능별 페이지 PATH	설명
/togethers	구인 게시물 리스트 페이지
/togethers/write	구인 게시물 작성 페이지
/togethers/:postId	구인 게시물 상세 열람 페이지
[Option 기능]	등급, 유저 프로필 Role 연관도에 따라 리스트 순위화

주요 기능 설명

① 프로젝트 팀원 구인하기 기능 (게시물 형태)

구인 게시물 리스트 페이지 기능 명세

- 필터: All, frontend | backend | design | android
- Option) 등급에 따라 팀플 모집 게시물 노출 우선순위 올려주기 등 순위화 가능하다면 구현

프로젝트 상세 페이지 기능 명세

- 희망인원 (현재인원수)/(정원)
- userId $\neq$ createUserId $\Rightarrow$ 신청하기 버튼 노출
- userId == createUserId $\Rightarrow$ 수정하기 버튼 노출
- Option) 조회수(상세 페이지 클릭수), 찜하기

프로젝트(구인) 수정 페이지 기능 명세

- 모집 상태 수정(default=모집중): 모집중 | 모집마감

주요 기능 설명

① 프로젝트 팀원 구인하기 기능 (게시물 형태)

구인 게시물 생성(수정도 동일) 페이지 기능 명세

- 파트 타입 (직무 타입): frontend | backend | design | android
- 참여 단계 (단일 셀렉터로 구현: 기획부터 | 개발부터 | 유지보수)
 - 개발부터
 - 기획서(생성자가 공개하고 싶은 만큼만 첨부, PDF) 파일 첨부란 노출
 - 기획서 공개범위 선택: 전체공개 | 승인한 사람만
 - 승인된 사람만 체크할 경우 블라인드 처리
 - 유지보수
 - 프로젝트 설명 등 생성자가 공개하고 싶은 만큼만 첨부, PDF) 파일 첨부란 노출
 - 프로젝트 설명 파일 공개범위 선택: 전체공개 | 승인한 사람만
 - 승인된 사람만 체크할 경우 블라인드 처리

주요 기능 설명

F-APP-001 프로젝트 참여 신청 생성

트리거: User Action

사전 조건:

- 사용자는 로그인 상태여야 한다.
- 프로젝트는 모집중 상태여야 한다
- 사용자는 해당 구인의 작성자가 아니어야 한다.

사후 조건:

- 참여 신청이 대기 상태로 저장된다.
- 구인 작성자에게 신청 알림이 생성된다.

인수 기준:

1. Given 비작성자 로그인 사용자가 모집중 구인에 신청할 때,
When 신청 조건을 충족하면, Then **PENDING** 신청이
생성된다.
2. Given 작성자가 본인 구인에 신청할 때, Then 신청은
차단된다.
3. Given 동일 사용자의 기존 대기 신청이 있을 때, Then 중복
신청은 차단된다.

처리 로직:

1. 인증 상태를 검증한다.
2. 구인 게시글을 조회한다.
3. if 구인이 없으면 신청을 중단한다.
4. if 신청자가 구인 작성자이면 신청을 차단한다.
5. if 구인 상태가 **CLOSED**이면 신청을 차단한다.
6. if 현재 승인 인원이 희망 인원 이상이면 신청을 차단한다.
7. if 동일 사용자의 대기 또는 승인 신청이 이미 있으면
신청을 차단한다.
8. 신청 상태를 **PENDING**으로 생성한다.
9. 구인 작성자에게 알림을 생성한다.

주요 기능 설명

F-APP-001 프로젝트 참여 신청 생성

입력값:

항목	타입	필수	제약조건	비고
postId	string	필수	존재하는 프로젝트 ID	신청대상
applicantId	string	필수	인증 사용자	신청자
message	string	선택	500자 이하	신청 메시지

출력값:

항목	타입	설명
applicationId	string	신청 ID
applicationStatus	enum	PENDING
notificationCreated	boolean	알림 생성 여부

주요 기능 설명

F-APP-001 프로젝트 참여 신청 생성

예외처리:

예외 코드	발생 조건	처리 방안	사용자 메시지
APP-001-UNAUTHENTICATED	비로그인 신청	신청 차단	로그인이 필요한 기능입니다.
APP-001-POST-NOT-FOUND	구인 없음	신청 중단	구인 정보를 찾을 수 없습니다.
APP-001-AUTHOR-NOT-ALLOWED	작성자 본인 신청	신청 차단	본인이 만든 구인에는 신청할 수 없습니다.
APP-001-CLOSED	모집마감 상태	신청 차단	모집이 마감된 구인입니다.
APP-001-FULL	정원 초과	신청 차단	모집 인원이 가득 찼습니다.
APP-001-DUPLICATED	중복 신청	신청 차단	이미 신청한 구인입니다.
APP-001-CREATE-FAILED	신청 저장 실패	신청 중단	참여 신청에 실패했습니다.

주요 기능 설명

② 프로젝트 참여 신청 관리 대시보드

구인 게시글 작성자는 본인이 등록한 프로젝트에 들어온 참여 신청을 한곳에서 확인하고 관리할 수 있다. 신청자의 기본 프로필을 바탕으로 승인 여부를 판단할 수 있으며, 신청자는 본인의 참여 신청 및 승인 여부를 확인할 수 있다.

- responses 페이지는 `userId == createUserId` 대함
- requests 페이지는 `userId ≠ createUserId`에 대함
- 공통 리스트 필터: `pending` | `approved` | `rejected`

기능별 페이지 PATH	설명
/profile/responses	생성한 게시물에 대해 신청한 신청자들을 승인 및 반려할 수 있는 페이지
/profile/requests	본인이 신청한 프로젝트에 대해 승인 및 반려 여부를 확인할 수 있는 페이지

주요 기능 설명

③ 로그인/회원 가입 및 포트폴리오 기능

Github 간편 로그인 API을 통해 회원 가입 및 로그인한다.

유저의 Github ID를 사용하여 포트폴리오 페이지에서 Github에서 지원하는 API를 통해 포트폴리오를 생성한다. Github API를 통한 포트폴리오는 기본 포트폴리오 템플릿으로, 이 외의 추가적인 정보도 유저의 직접 입력을 통해 덧붙일 수 있도록 한다.

기능별 페이지 PATH	설명
/login	생성한 게시물에 대해 신청한 신청자들을 승인 및 반려할 수 있는 페이지
/profile	본인이 신청한 프로젝트에 대해 승인 및 반려 여부를 확인할 수 있는 페이지

주요 기능 설명

③ 로그인/회원 가입 및 포트폴리오 기능

포트폴리오 기능

유저 기본 정보	GET	• <code>/users/{username}</code> • https://api.github.com/users/dayeon-choi
레포지토리 목록	GET	• <code>/users/{username}/repos</code> • https://api.github.com/users/dayeon-choi/repos
	GET	• <code>/repos/{owner}/{repo}/languages</code> • https://api.github.com/repos/dayeon-choi/duzzle/languages
커밋 이력	GET	• <code>/repos/{owner}/{repo}/commits</code>
기여도 이벤트	GET	• <code>/users/{username}/events</code>

주요 기능 설명

F-PROF-003: 나의 구인 목록 조회

트리거: User Action

사전 조건:

- 사용자는 인증 상태여야 한다.

사후 조건:

- 데이터 상태 변경 없음.

인수 기준:

1. Given 로그인 사용자가 나의 구인을 요청했을 때, Then 작성 구인과 참여 구인이 반환된다.
2. Given 작성 구인과 참여 구인이 모두 없을 때, Then 빈 목록이 반환된다.

처리 로직:

1. 인증 상태를 검증한다.
2. 사용자가 작성한 구인 목록을 조회한다.
3. 사용자가 승인 참여자로 등록된 구인 목록을 조회한다.
4. 중복 항목이 있으면 하나의 항목으로 병합한다.
5. 페이지 조건에 맞게 반환한다.

주요 기능 설명

F-PROF-003: 나의 구인 목록 조회

입력값:

항목	타입	필수	제약조건	비고
userId	string	필수	인증 사용자	조회 대상
page	number	선택	1 이상	기본값 1
size	number	선택	1~50	기본값 20

출력값:

항목	타입	설명
posts	array	나의 구인 목록
createdCount	number	내가 만든 구인 수
joinedCount	number	내가 참여한 구인 수
totalCount	number	전체 수

주요 기능 설명

F-PROF-003: 나의 구인 목록 조회

예외처리:

예외 코드	발생 조건	처리 방안	사용자 메시지
PROF-003-UNAUTHENTICATED	비로그인 조회	조회 차단	로그인이 필요한 기능입니다.
PROF-003-INVALID-PAGING	페이지 조건 오류	기본값 적용 또는 오류 반환	조회 조건이 올바르지 않습니다.
PROF-003-LOAD-FAILED	조회 실패	오류 반환	나의 구인 목록을 불러오지 못했습니다.

주요 기능 설명

④ 참고 프로젝트 주제 페이지

참고 프로젝트 주제를 아래 공공데이터 활용 창업경진대회 API를 연동하여 참고할 수 있도록 함.

- <https://www.data.go.kr/data/15153117/fileData.do>

기능별 페이지 PATH	설명
/references	주제를 참고할 수 있는 레퍼런스 페이지

주요 기능 설명

③ 포트폴리오 관리, 확인 및 공유 기능

: 프로필 페이지 내 Github 프로필 연동을 통한 스택 및 스펙을 확인할 수 있는 기능이다.

회원가입 시 Github 간편 로그인 API 사용, 해당 정보를 사용해 포트폴리오 생성한다. UI에는 그래프를 사용한다.

기능별 페이지 PATH	명세
/profile	프로필 페이지. 프로필 이미지, 아이디, 사용자명 등 기본 사용자 정보를 확인
/profile/portfolio	GitHub 포트폴리오 페이지. GitHub 기반 사용자 정보, 레포지토리, 사용 언어, 커밋 및 기여도 정보를 확인
/profile/notification	참여 신청, 신청 승인, 신청 반려 등 알림 정보를 확인

주요 기능 설명

F-PROF-001: 내 프로필 조회

트리거: User Action

사전 조건:

- 사용자는 인증 상태여야 한다.
- 사용자 프로필이 존재해야 한다

사후 조건:

- 데이터 상태 변경 없음.

인수 기준:

1. Given 로그인 사용자가 프로필을 요청했을 때, When 프로필이 존재하면, Then 본인의 프로필 정보가 반환된다.
2. Given 프로필 정보가 없을 때, Then PROF-001-NOT-FOUND 예외가 발생한다.

처리 로직:

1. 인증 상태를 검증한다.
2. userId에 해당하는 프로필을 조회한다.
3. GitHub 연결 상태를 조회한다.
4. 프로필 기본 정보와 연결 상태를 반환한다.

주요 기능 설명

F-PROF-001: 내 프로필 조회

입력값:

항목	타입	필수	제약조건	비고
userId	string	필수	인증 사용자	조회 대상

출력값:

항목	타입	설명
userId	string	사용자 ID
username	string	서비스 노출 사용자명
profileImageUrl	string	프로필 이미지
githubUsername	string	GitHub 사용자명
githubLinked	boolean	GitHub 연결 여부

주요 기능 설명

F-PORT-001: 내 GitHub 포트폴리오 조회

입력값:

항목	타입	필수	제약조건	비고
userId	string	필수	인증 사용자	조회 대상
filter	enum	필수	ALL, CREATED, JOINED	필터 조건
page	number	선택	1 이상	기본값 1
size	number	선택	1~50	기본값 20

출력값:

항목	타입	설명
filter	enum	적용된 필터
posts	array	필터 적용 결과
totalCount	number	결과 수

주요 기능 설명

F-PORT-001: 내 GitHub 포트폴리오 조회

예외처리:

예외 코드	발생 조건	처리 방안	사용자 메시지
PORT-001-UNAUTHENTICATED	비로그인 조회	조회 차단	로그인이 필요한 기능입니다.
PORT-001-GITHUB-NOT-LINKED	GitHub 연결 없음	조회 차단	GitHub 계정 연결이 필요합니다.
PORT-001-NO-SNAPSHOT	포트폴리오 스냅샷 없음	동기화 필요 상태 반환	포트폴리오 정보를 갱신해주세요.
PORT-001-LOAD-FAILED	조회 실패	오류 반환	포트폴리오 정보를 불러오지 못했습니다.

주요 기능 설명

F-PROF-001: 내 프로필 조회

예외처리:

예외 코드	발생 조건	처리 방안	사용자 메시지
PROF-001-UNAUTHENTICATED	비로그인 조회	조회 차단	로그인이 필요한 기능입니다.
PROF-001-NOT-FOUND	프로필 없음	조회 중단	프로필 정보를 찾을 수 없습니다.
PROF-001-LOAD-FAILED	조회 실패	오류 반환	프로필 정보를 불러오지 못했습니다.

주요 기능 설명

F-PROF-002: 사용자명 수정

트리거: User Action

사전 조건:

- 사용자는 인증 상태여야 한다.
- 사용자 프로필이 존재해야 한다

사후 조건:

- 사용자 노출명이 변경된다.

인수 기준:

1. Given 로그인 사용자가 중복되지 않는 사용자명을 입력했을 때, When 수정을 요청하면, Then 사용자명이 변경된다.
2. Given 중복 사용자명을 입력했을 때, Then 수정은 차단된다.
- 3 Given 형식에 맞지 않는 사용자명을 입력했을 때, Then 수정은 차단된다.

처리 로직:

1. 인증 상태를 검증한다.
2. username 형식을 검증한다.
3. username 중복 여부를 확인한다.
4. if 중복이면 수정을 차단한다.
5. 현재 사용자의 username을 변경한다.
6. 변경 이력을 기록한다.

주요 기능 설명

F-PROF-002: 사용자명 수정

입력값:

항목	타입	설명	항목	타입
userId	string	사용자 ID	userId	string
username	string	변경된 사용자명	username	string

출력값:

항목	타입	설명
userId	string	사용자 ID
username	string	변경된 사용자명
updatedAt	datetime	수정 시각

주요 기능 설명

F-PROF-002: 사용자명 수정

예외처리:

예외 코드	발생 조건	처리 방안	사용자 메시지
PROF-002-UNAUTHENTICATED	비로그인 수정	수정 차단	로그인이 필요한 기능입니다.
PROF-002-INVALID-FORMAT	사용자명 형식 오류	수정 차단	사용자명 형식을 확인해주세요.
PROF-002-DUPLICATED	사용자명 중복	수정 차단	이미 사용 중인 사용자명입니다.
PROF-002-UPDATE-FAILED	저장 실패	수정 차단	사용자명 수정에 실패했습니다.

주요 기능 설명

F-PORT-001: 내 GitHub 포트폴리오 조회

트리거: User Action

사전 조건:

- 사용자는 인증 상태여야 한다.
- GitHub 계정이 연결되어 있어야 한다.

사후 조건:

- 데이터 상태 변경 없음.

인수 기준:

1. Given GitHub 연결 사용자가 포트폴리오를 요청했을 때,
When 스냅샷이 존재하면, Then 포트폴리오 정보가 반환된다.
2. Given 스냅샷이 없을 때, Then `syncRequired=true`가 반환된다.

처리 로직:

1. 인증 상태를 검증한다.
2. GitHub 연결 상태를 확인한다.
3. 저장된 포트폴리오 스냅샷을 조회한다.
4. if 스냅샷이 없으면 `syncRequired=true`로 반환한다.
5. 스냅샷이 있으면 사용자 기본 정보, 레포지토리, 언어, 커밋, 기여도 요약을 반환한다.

기술 스택

기술 스택

백엔드/프론트엔드 구성

React.js, JavaScript  

Database

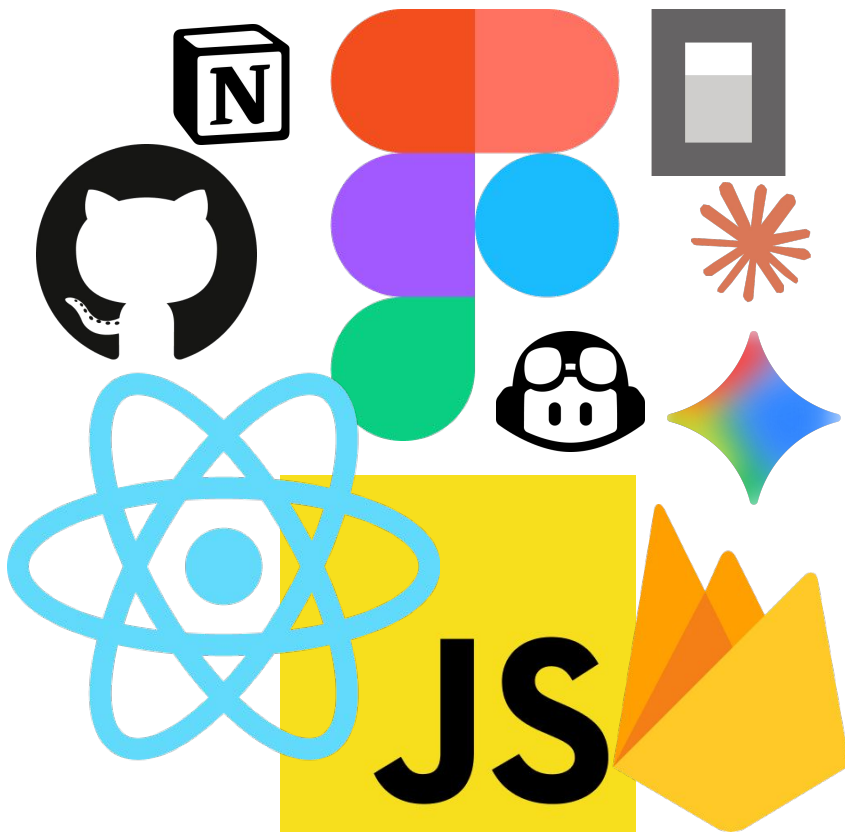
Firebase 

협업

Github, Notion, Discord   

AI

Copilot github, Claude,    
OpenCode, Gemini, Codex



개발 일정

개발 일정

세부 개발 내용	과제 수행 기간					
	5월 4주		6월 1주		6월 2주	
웹 퍼블리싱						
홈 / 로그인 기능 구현						
구인 기능 구현						
포트폴리오 기능 구현						
참고 주제 기능 구현						
프로젝트 신청 관리 대시보드 구현						
권한/필터 조건 반영 구현						
QA 통합 테스트 및 보고서 작성						

기능구현

페이지/기능 커버리지 매트릭스

#	페이지(Code)	라우트	User Flow
1	Home	/	F1, F2
2	Login	/login	F1
3	ReferenceList	/references	F2, F5
4	MyScrappedReferences	/profile/references/my-scrap	F2, F6
5	RetrospectiveList	/memoirs	F6
6	RetrospectiveDetail	/memoirs/:id	F6
7	RetrospectiveWrite	/memoirs/write	F6
8	RetrospectiveEdit	/memoirs/:id/edit	F6
9	MyRetrospectives	/profile/memoirs/my-article	F6

페이지/기능 커버리지 매트릭스

#	페이지(Code)	라우트	User Flow
10	ProjectList	/togethers	F3, F4
11	ProjectDetail	/togethers/:id	F3, F4
12	ProjectCreate	/togethers/write	F3
13	ProjectEdit	/togethers/:id/edit	F3
14	MyProjects	/profile/togethers	F3
15	ResponseList	/profile/responses	F3
16	RequestList	/profile/requests	F4
17	Profile	/profile	F7
18	Portfolio	/profile/portfolio	F7

페이지/기능 커버리지 매트릭스

#	페이지(Code)	라우트	User Flow
19	Notifications	/profile/notifications	F8
20	404 (*)	/togethers/:id	F8
-	Header · NotificationDropdown · ReferenceFormModal · ProtectedRoute · 반응형		F1/F5/F8

[User Flow] 001 – 진입 & 인증



로그인 인증 플로우

플로우 및 설명: router.jsx의 ProtectedRoute를 통해 자동 리다이렉트 처리

- [둘러보기 / 헤더 로그인 버튼] → 로그인 페이지(/login, ②) GitHub OAuth → 인증 성공 → ③ 원래 가려던 페이지 (redirectUrl)로 복귀, 및 헤더

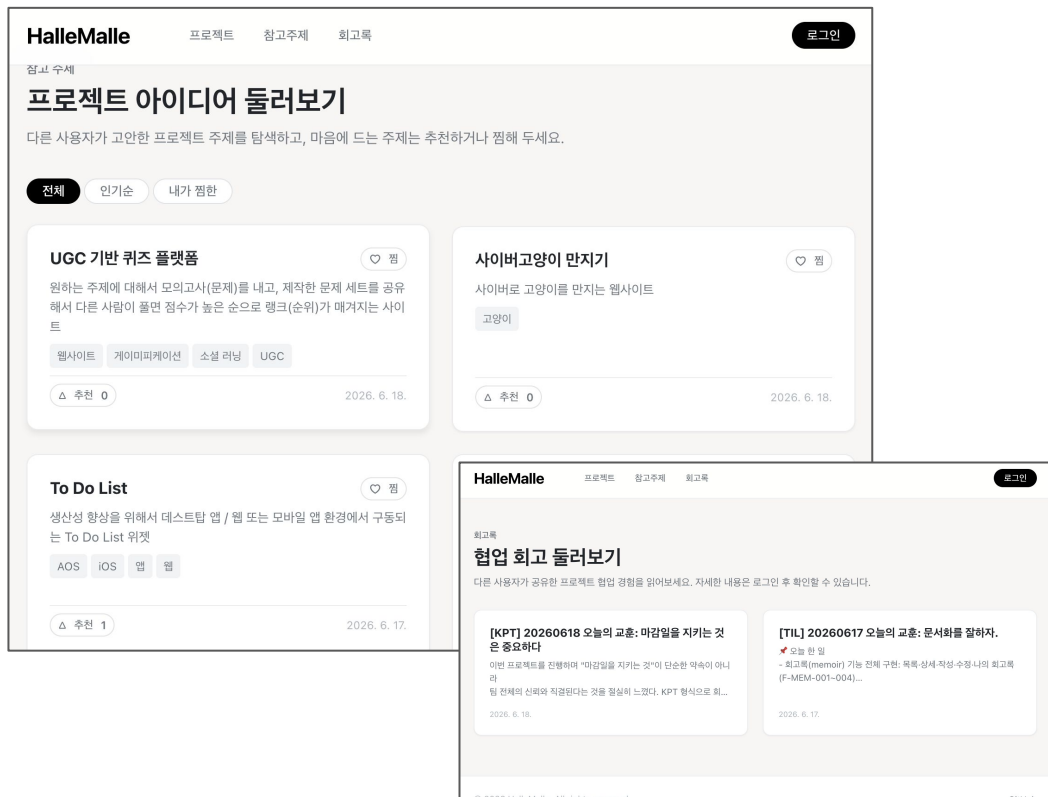
hallemalle.vercel.app/login?returnUrl=%2Fmemoirs%2Fbujypden75emg3FdvaDc

- 보호 페이지 직접 접근 → ProtectedRoute가 /login으로 리다이렉트 구현

비로그인 방문자 탐색 (게스트 열람)

- 로그인(/login), 참고주제(/references), 회고록(/memoirs)은 비로그인 접근 가능

[User Flow] 002 – 비로그인 방문자 탐색



플로우 및 설명: router.jsx의 ProtectedRoute를 통해 자동 리다이렉트 처리

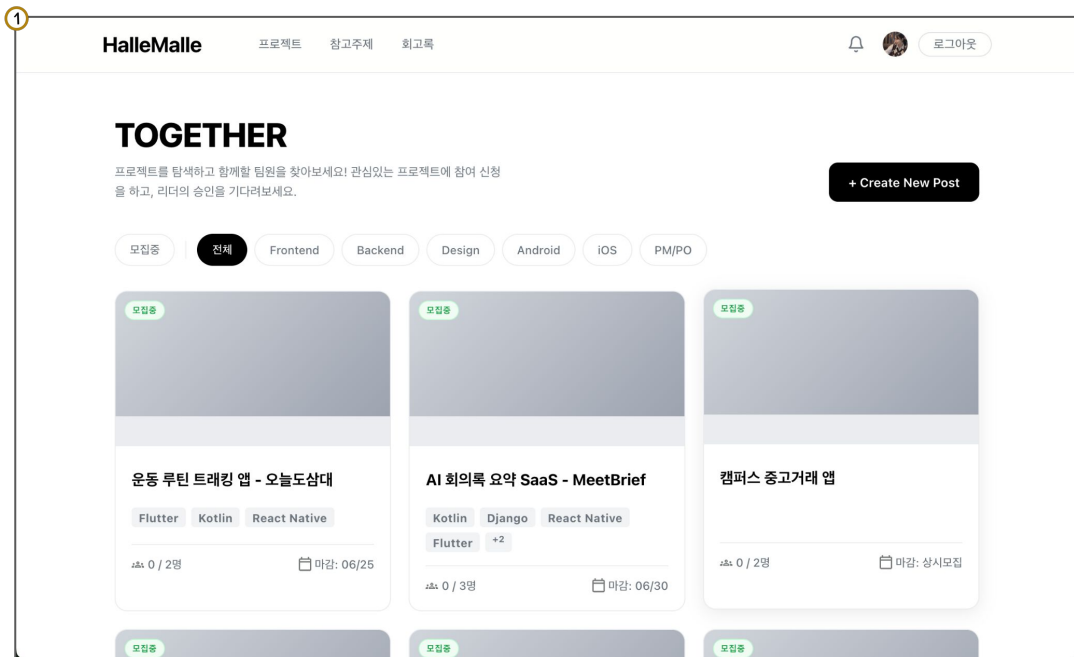
비로그인 방문자 탐색 (게스트 열람)

- 로그인(/login), 참고주제(/references), 회고록(/memoirs)은 비로그인 접근 가능

탐색 시 로그인 페이지 리다이렉트 케이스

- 참고주제: 찜, 추천, 내가 추천 클릭 시 로그인 페이지 리다이렉트
- 회고록: 카드 컨테이너 클릭 시 로그인 페이지 리다이렉트

[User Flow] 003 – 팀원 구하기 (구인자 / 리더 관점)



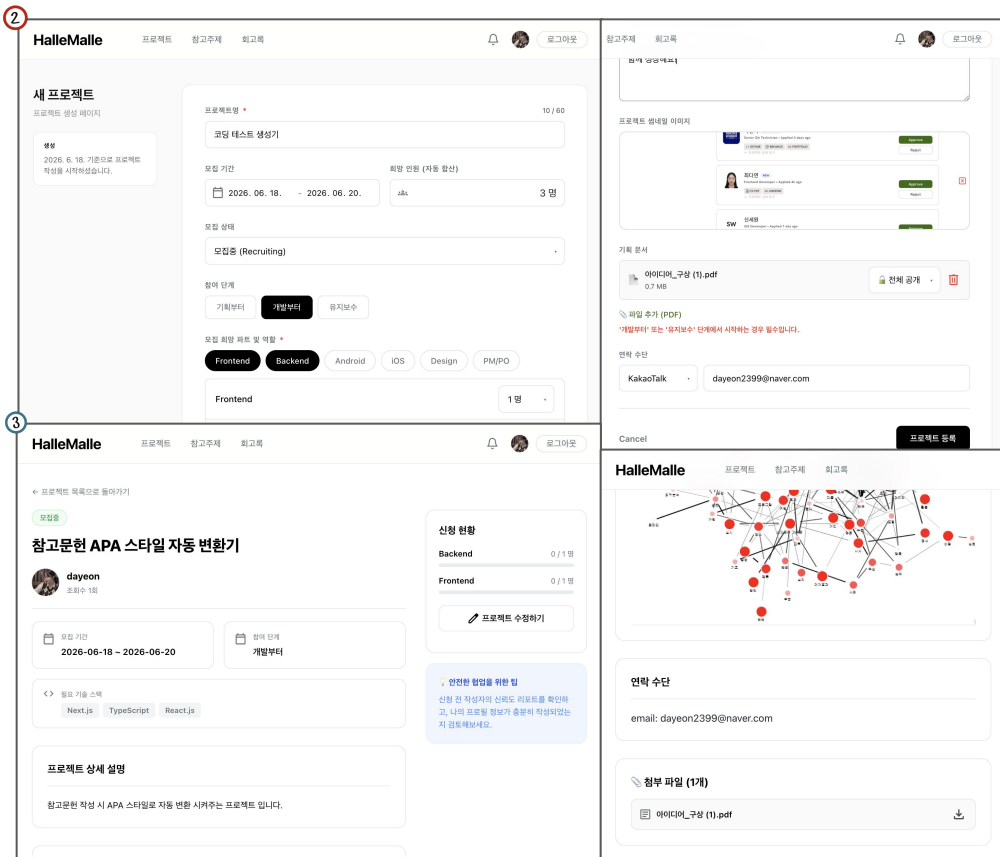
게시물 작성자(구인자) 플로우

- 프로젝트 목록(/togethers)
 - 새 글 작성(/togethers/write)
 - 내 공고 상세(/togethers/:id) [모집중]
 - 내 구함글 관리(/profile/togethers)
 - 수정(/togethers/:id/edit)
 - 받은 신청 관리(/profile/responses)
 - 승인 / 반려
 - 승인자에게 알림 발송 · 승인자 전용 첨부파일 공개

기능 설명 (구인글 CRUD)

- Read List(①):** 생성일 기준 DESC 정렬해 together, together_roles, together_tech_stack 컬렉션 사용해 리스트 형태로 응답

[User Flow] 003 – 팀원 구하기 (구인자 / 리더 관점)



기능 설명 (구인글 CRUD)

- **CREATE(2):**
 formData, userId로 together, together_roles, together_tech_stack, planning_documents 컬렉션에 데이터 분할 생성.
 - 썸네일 및 PDF 문서는 클라이언트단에서 Base64 문자열로 변환 후 Firestore에 직접 바인딩 처리.
 - 발급된 고유 post_id를 외래키(FK) 구조로 활용하여 하위 서브 컬렉션 데이터 정합성 확립.
- **READ(3):** 메인 공고 및 3개의 서브 컬렉션을 병렬 쿼리하여 유기적 데이터 바인딩 구현.

[User Flow] 003 – 팀원 구하기 (구인자 / 리더 관점)

4

HalleMalle 프로젝트 참고주제 회고록

프로젝트 수정

기본 프로젝트 정보 수정

수정 상태

작성자가 본인 인증 완료 후 내용을 수정하고 있습니다.

프로젝트명 * 19 / 60

참고문헌 APA 스타일 자동 변환기

모집 기간 희망 인원 (자동 합산)

2026. 06. 18. - 연도, 월, 일. 2인 0명

모집 상태

모집중 (Recruiting)

참여 단계

기획부터 개발부터 유지보수

모집 희망 파트 및 역할 *

Frontend Backend Android iOS Design PM/PO

희망 기술 선택

JavaScript TypeScript React.js Next.js Vue.js Nuxt.js

Node.js Express NestJS Java SpringBoot Python

Django FastAPI Go Swift Kotlin Flutter

기능 설명 (구인글 CRUD)

- **UPDATE (④):**

postId, data, uid를 활용한 기존 게시글 및 서브 컬렉션 유지적 갱신.

- 메인 공고 정보는 updateDoc을 활용해 원자적으로 변경 사항만 반영하여 리소스 낭비 방지.
- 변경 및 가변성이 잦은 역할군, 기술 스택, 첨부 파일 데이터는 기존 서브 데이터를 전체 삭제(deleteDoc) 처리한 후 새로운 data 스펙으로 일괄 재등록하여 정적 정합성 유지.

- **DELETE (⑤):**

단순 공고글 삭제에 그치지 않고, post_id 기준 하위 서브 컬렉션 일괄 삭제.

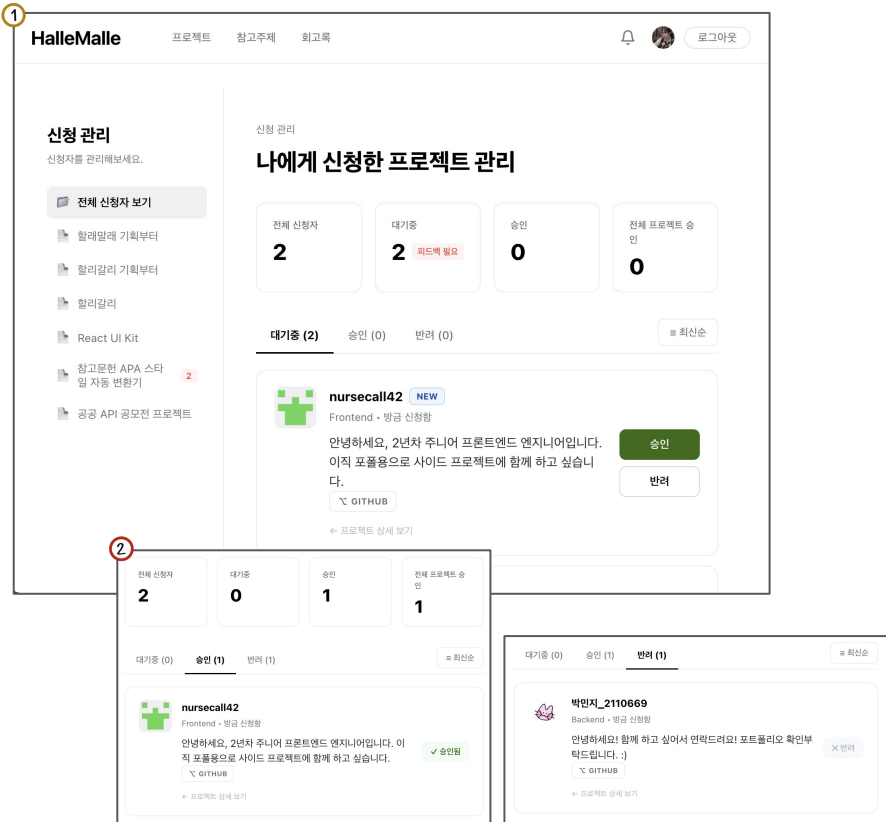
- 모든 팀원 지원서 데이터(applicants) 및 개별 지원자가 업로드했던 첨부파일 데이터(applicant_attachments) 까지 식별하여 데이터베이스 내에서 연쇄적으로 완전 삭제 처리.

5

Cancel

프로젝트 삭제

[User Flow] 003 – 팀원 구하기 (구인자 / 리더 관점)



기능 설명 (승인 & 반려)

프로젝트 개설자(Owner)가 지원자의 참여 신청을 심사하여 승인 또는 반려하는 기능.

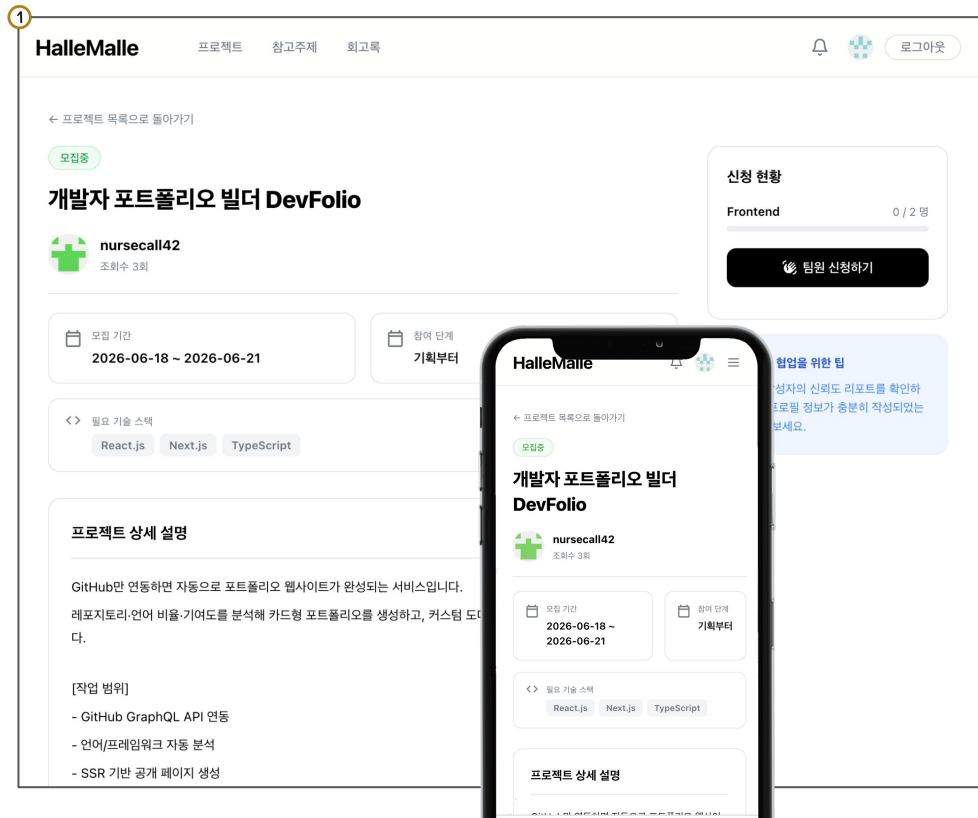
• 승인

- 상태 변경: 지정된 신청서 고유 ID(applicantId)의 status 필드 값을 approved로 업데이트.
- 이력 저장: 심사가 완료된 서버 타임스탬프(serverTimestamp())를 reviewed_at 필드에 동기화.

• 반려

- 상태 변경: 지정된 신청서 고유 ID(applicantId)의 status 필드 값을 rejected로 업데이트.
- 이력 저장: 심사가 완료된 서버 타임스탬프(serverTimestamp())를 reviewed_at 필드에 동기화.

[User Flow] 004 – 프로젝트 참여 신청 (지원자 관점)



게시물 작성자(구인자) 플로우

- 로그인
 - 프로젝트 목록(/togethers)
 - 상세(/togethers/:id)
 - 팀원 신청하기 버튼 클릭
 - 모달 폼대로 역할 선택 후 신청
 - 작성자에게 알림 전달
 - 내 신청 현황(/profile/requests) [승인/반려 확인 · 신청 취소]
 - 승인되면 알림 수신
 - 승인자 전용 첨부 열람

조회수 증가 기능 (READ 시)

- 지원자가 프로젝트 상세 페이지 (getProjectDetail)에 진입시 클라이언트 생명주기(마운트 시점)에서 독립적으로 호출
- Firestore의 view_count 필드를 원자적 (increment(1))으로 즉시 업데이트함.

[User Flow] 004 – 프로젝트 참여 신청 (지원자 관점)

2

팀원 신청하기

신청 역할 *

Frontend
0/2

포트폴리오 링크 (최대 5개)

GitHub

https://github.com/oo-dada009

+ 링크 추가

신청 메시지 (선택)

포트폴리오 흥미롭습니다! 도전적인 주제라 신청했습니다. React, Typescript, Nuxt 모두 선호합니다.

64 / 500

취소

신청하기

3

팀원 신청하기 검토 중

신청서 검토 중입니다. 결과를 기다려주세요.

신청 역할 *

Frontend
0/2

포트폴리오 링크 (최대 5개)

GitHub

https://github.com/oo-dada009

신청 메시지 (선택)

포트폴리오 흥미롭습니다! 도전적인 주제라 신청했습니다. React, Typescript, Nuxt 모두 선호합니다.

64 / 500

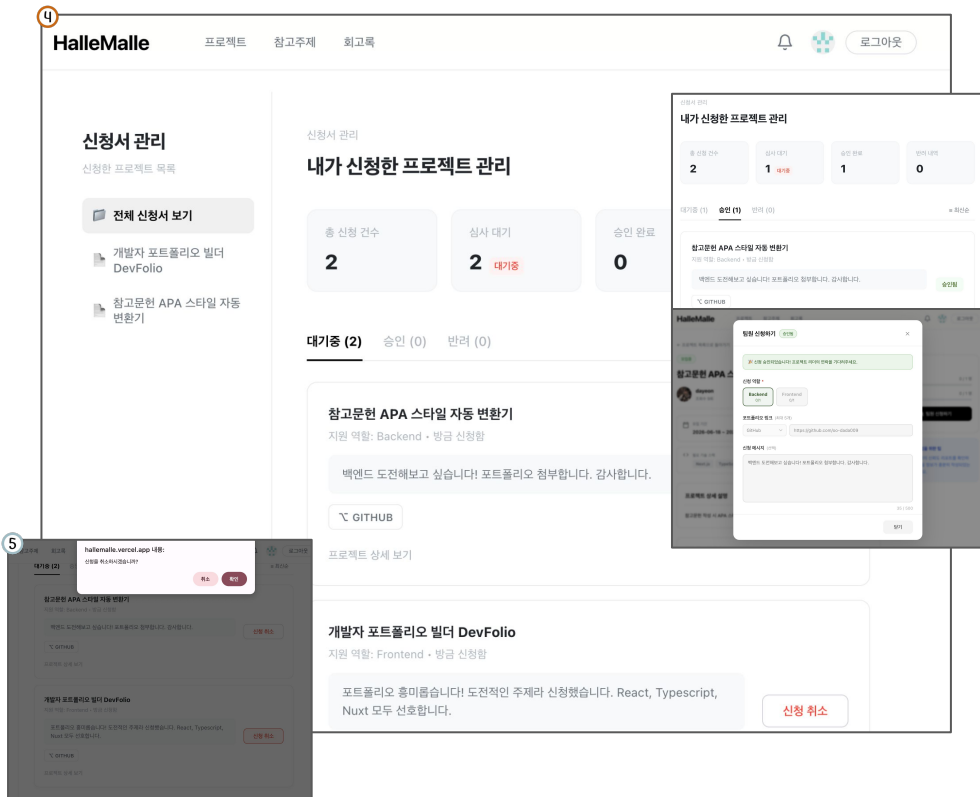
신청 취소

닫기

신청 기능 설명 (CREATE)

- **플로우**
상세 페이지에서 신청 모달을 통해 지원자가 formData 입력 시 applicants 및 applicant_attachments 컬렉션에 지원 데이터를 생성하고 개설자에게 알림을 발송
- **중복 신청 방지**
이미 지원 이력이 있는지 데이터베이스를 getMyApplication하여 중복 지원 시 에러 처리
- **필수값 검증**
Role(역할군)이 누락되었을 경우 에러로 예외 처리.

[User Flow] 004 – 프로젝트 참여 신청 (지원자 관점)



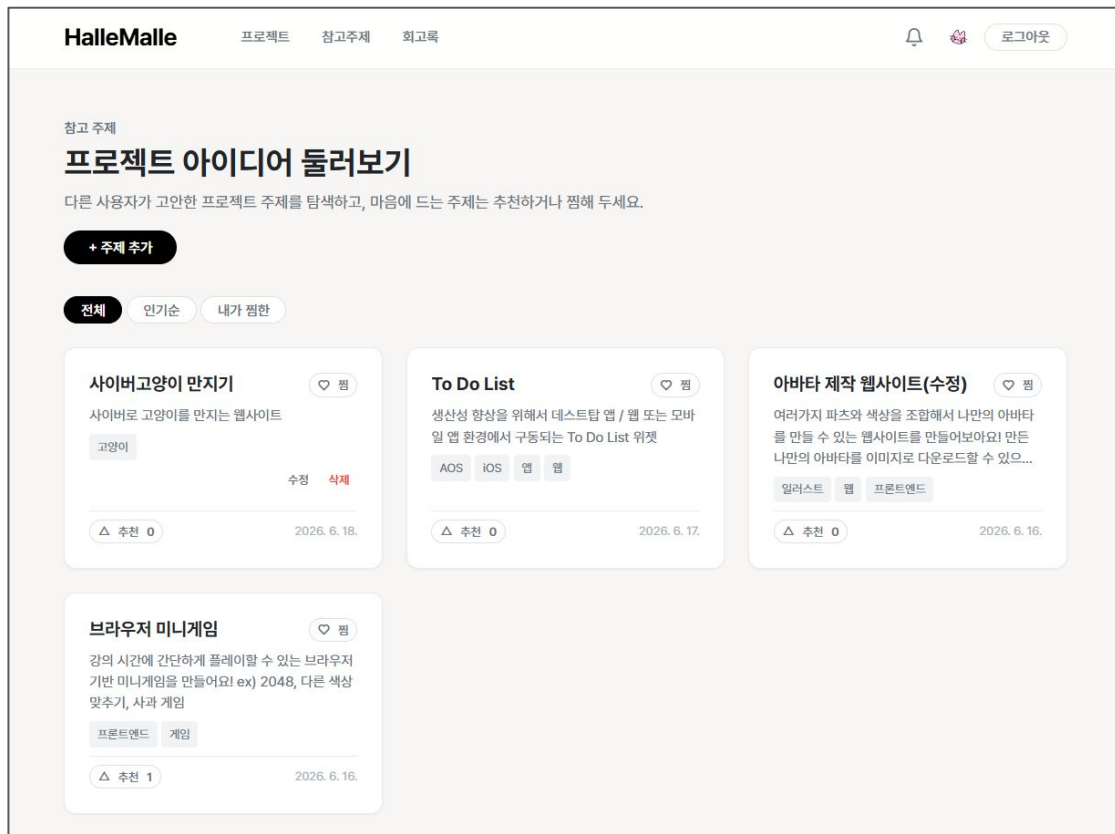
신청 결과 및 프로젝트 조회 기능 (READ)

- 지원자가 자신이 신청한 모든 지원서의 진행 상황(대기, 승인, 반려)과 대상 프로젝트의 정보를 한눈에 모니터링하는 기능.
- 지원자 고유 식별 키(uid) 기반으로 applicants 컬렉션 확인, 각 지원서에 매핑된 외래키(post_id)를 통해 togethers 컬렉션의 원본 공고 데이터(제목, 썸네일, 모집 상태 등)를 병렬로 조인

신청 취소 기능 (DELETE)

- 지원자가 자신이 제출한 참여 신청을 철회하고 관련 데이터를 데이터베이스에서 완전히 삭제하는 기능.
- 대기(status == "pending") 상태일 때만 취소가 가능하도록 비즈니스 로직 제어

[User Flow] 005 – 참고주제 활용



1) 참고 주제 목록 조회 F-REF-001

- 기능 설명: 사용자들이 공유한 프로젝트 참고 주제를 목록(카드) 형태로 탐색한다. 별도 상세 페이지 없이 목록 중심으로 제공된다.

- 동작 과정:

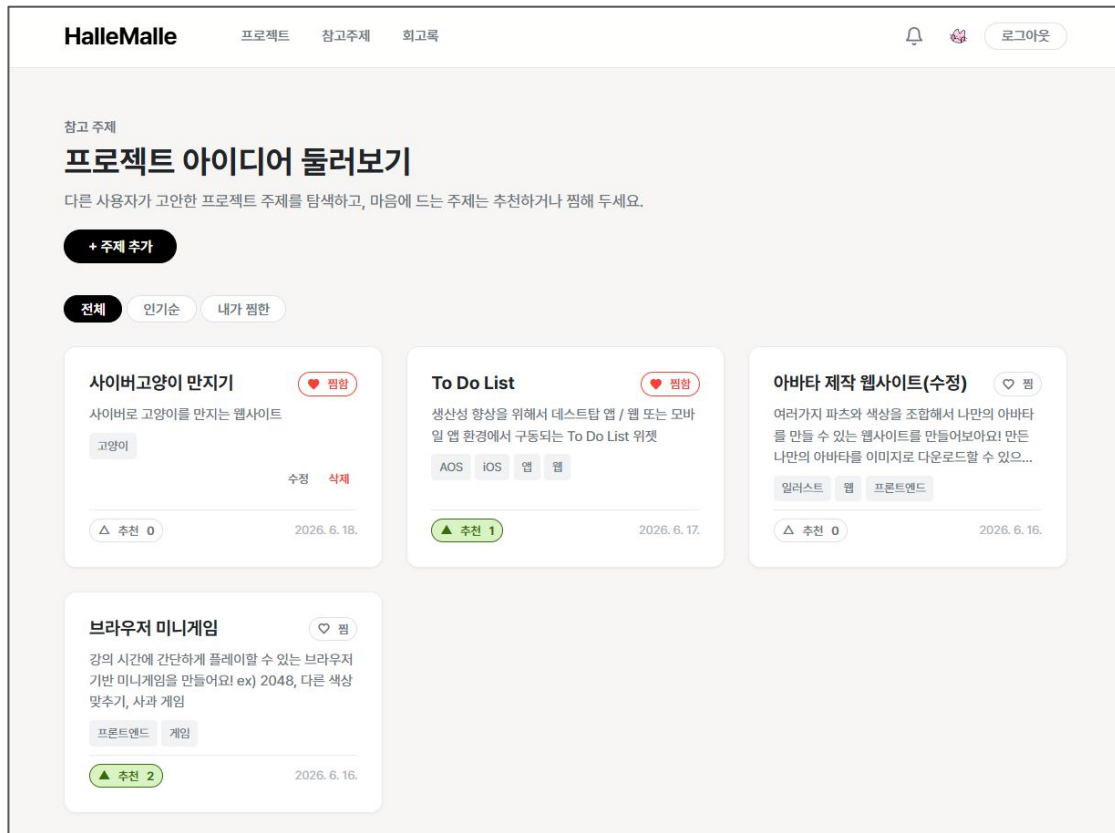
a. /references 진입 → 참고 주제 목록을 불러온다.

b. 각 카드에 제목·설명·태그·추천 수를 표시한다.

c. 추천(Like) 수 기준 인기 정렬을 지원한다.

- 비로그인 열람 허용(액션 시 로그인 유도).

[User Flow] 005 – 참고주제 활용



2) 참고 주제 추천(Like) F-REF-008

- 기능 설명: 마음에 드는 주제를 추천해 공개 추천 수를 누적한다. (개인 보관인 '찜'과 별개)

- 동작 과정:

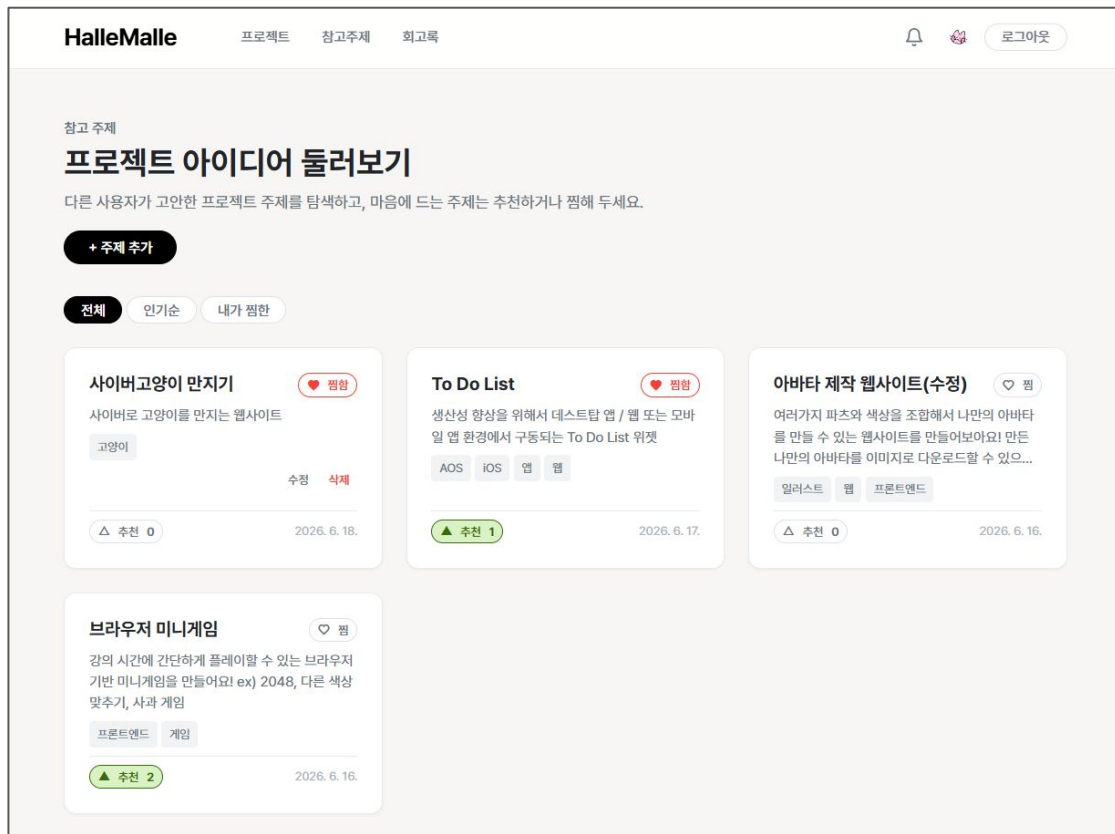
a. 카드의 추천 버튼 클릭 → LIKE/UNLIKE 토글.

b. 추천 생성/삭제와 주제의 likesCount를 트랜잭션으로 ± 1 갱신(동시 요청에도 일관성 유지).

c. 동일 사용자-주제 추천은 1건만 존재(중복 증가 없음).

- 비로그인 추천 시 → 로그인 페이지로 안내.

[User Flow] 005 – 참고주제 활용



3) 참고 주제 찜(스크랩) F-REF-002

- 기능 설명: 관심 주제를 개인 보관함에 담거나 해제한다.

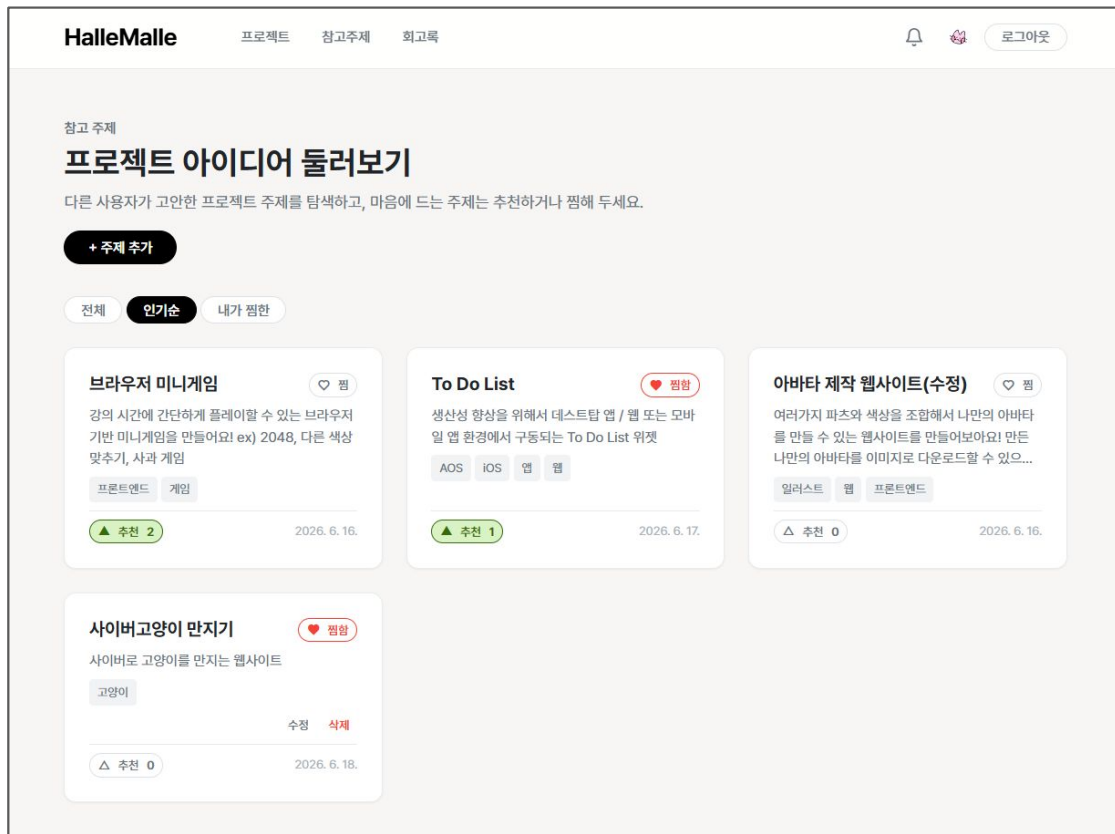
- 동작 과정:

a. 찜 버튼 클릭 → SCRAP/UNSCRAP.

b. 기존 찜 여부 확인 후 찜 정보를 생성/삭제 (중복 찜 방지).

c. 최종 찜 여부를 화면에 반영.

[User Flow] 005 – 참고주제 활용



4) 참고 주제 정렬 — 인기(추천)순 F-REF-001 · REQ-030

- 기능 설명: 참고 주제 목록을 인기순(추천 많은 순)으로 전환해 탐색한다. 추천(Like)이 많이 쌓인 주제를 상단에서 빠르게 발견할 수 있게 하는 정렬 기능이다. ('내가 찐한' 보기 탭도 동일 목록 위에서 함께 제공된다.)

- 동작 과정:

a. 목록 상단의 보기 탭에서 인기순(/ 내가 찐한)을 선택한다.

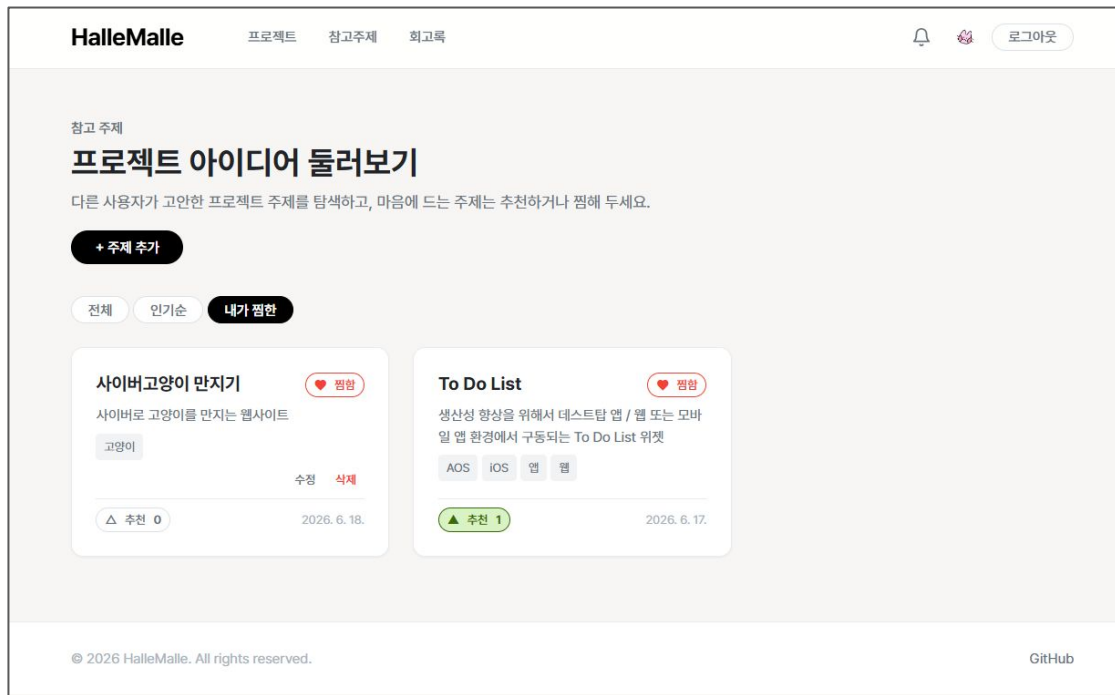
b. 선택한 탭이 조회 정렬로 매핑된다 — 인기순은 주제의 추천 수(likes_count) 내림차순, 최신순은 작성일(created_at) 내림차순으로 서버에서 정렬해 조회한다.

c. 정렬된 결과에 현재 사용자의 추천·찐 상태를 병합해 카드에 반영한다.

d. '내가 찐한' 탭은 최신순으로 불러온 뒤 본인이 찐한 항목만 클라이언트에서 필터링한다.

- 연계: 추천 수는 F-REF-008(추천 Like)에서 트랜잭션으로 누적된 값이라, 추천이 쌓일수록 인기순 상단에 노출된다.

[User Flow] 005 – 참고주제 활용

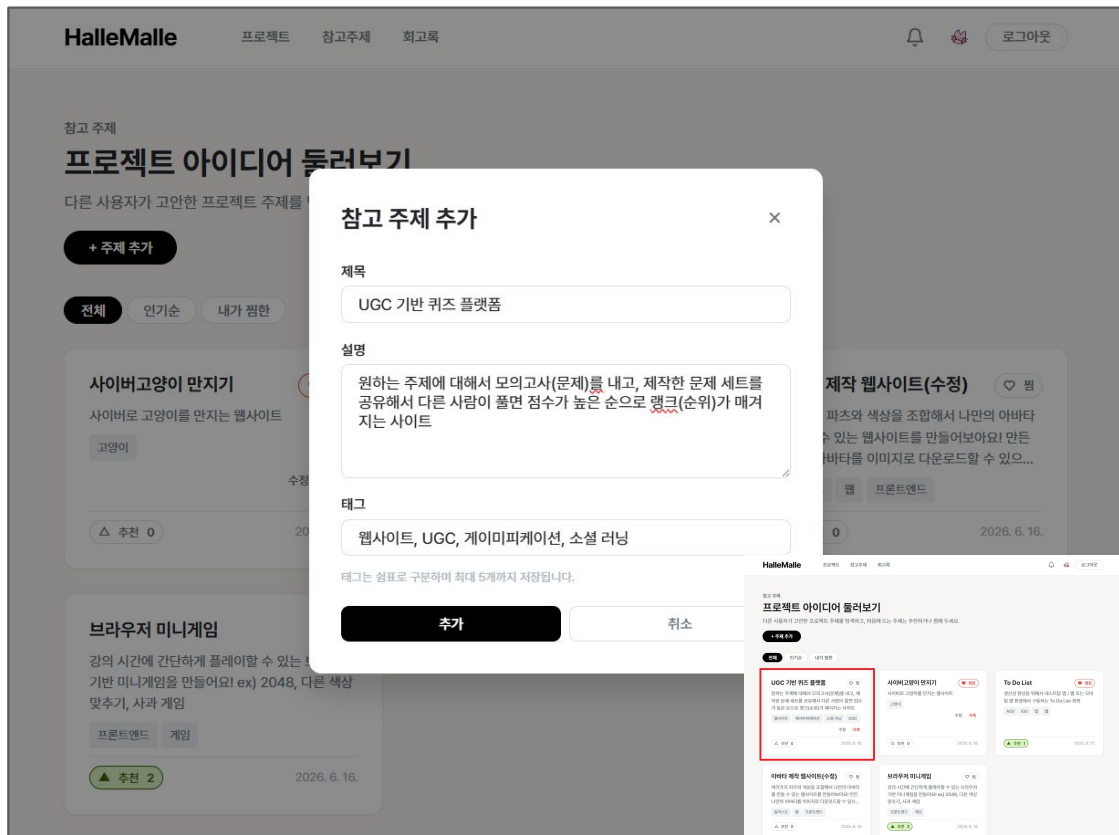


5) 내가 찜한 주제 모아보기 F-REF-003

- 기능 설명: 본인이 찜한 참고 주제만 모아 다시 확인한다.

- 동작 과정: /profile/references/my-scrap
→ 현재 사용자의 찜 목록만 조회(타인 데이터 미포함). 비어 있으면 빈 상태 안내.

[User Flow] 005 – 참고주제 활용



6) 참고 주제 작성·수정·삭제 F-REF-005 / 006 / 007

- 기능 설명: 브레인스토밍한 주제를 직접 작성하고, 작성자 본인만 수정·삭제할 수 있다. (작성/수정은 모달)

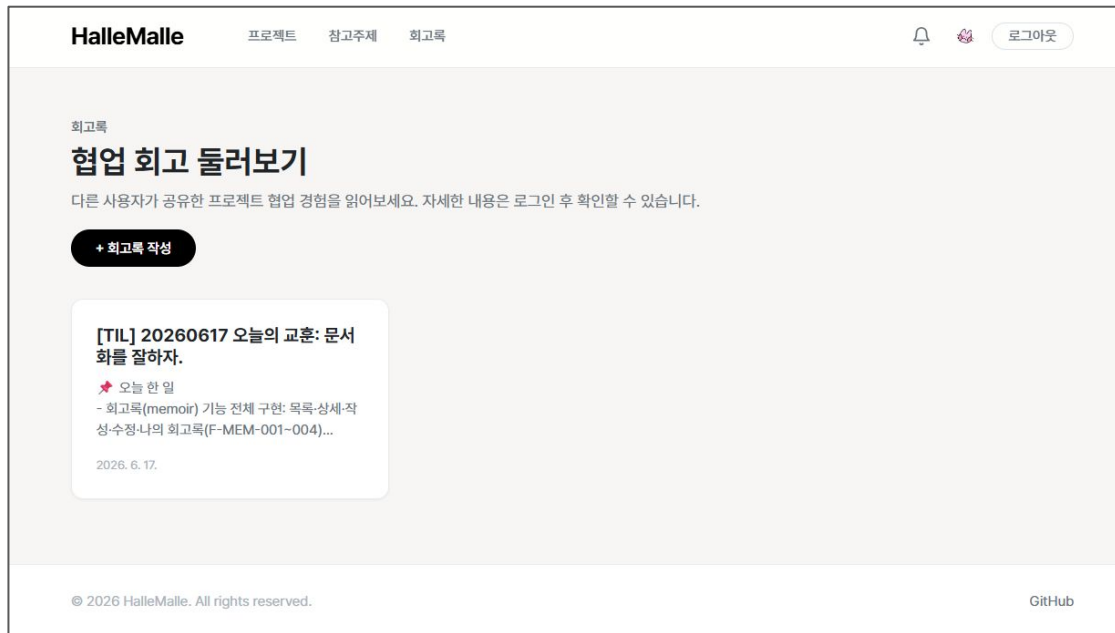
- 동작 과정:

a. 작성 모달에서 제목·설명·태그 입력 → 저장 시 sourceType=INTERNAL, likesCount=0으로 초기화.

b. 수정·삭제는 작성자 본인 검증(서버 Rules) 후 반영. 비작성자는 차단.

c. 삭제 시 연관 태그·추천·찜 데이터도 함께 정리.

[User Flow] 006 – 회고록 공유



1) 회고록 목록 조회 F-MEM-001

- 기능 설명: 다른 사용자가 공유한 전체 공개 회고(협업 경험)를 목록으로 둘러본다.
- 동작 과정: /memoirs 진입 → 공개 회고록만 최신순 조회 → 카드 목록 표시.
- 회고록 목록은 비로그인 열람 가능 / 상세는 로그인 게이트 안내

[User Flow] 006 – 회고록 공유

HalleMalle[프로젝트](#)[참고주제](#)[회고록](#)[로그아웃](#)

[< 회고록 목록으로](#)

[TIL] 20260617 오늘의 교훈: 문서화를 잘하자.

2026. 6. 17.

수정

★ 오늘 한 일

- 회고록(memoir) 기능 전체 구현: 목록-상세-작성-수정-나의 회고록(F-MEM-001~004)
- 비로그인 read 허용 + 추천/찜 로그인 유도(참고주제) 작업 마무리
- ready-check 게이트를 단계마다 통과시키며 Phase 1~6 진행

📌 오늘의 주제 — Retrospective vs Memoir, 그 사이의 고민

문서화는 정말 열심히 해왔는데, 정작 문서끼리 정책이 안 맞아서 하루 종일 이름 하나로 고민했다.

- URL은 memoirs, 코드 키는 Retrospective*, 그런데 컬렉션명이 문서마다 달랐다.
- ARCHITECTURE.md-04_TECHNICAL_DESIGN.md → retrospectives / camelCase(authorId)
- 배포된 firestore.rules → memoirs / snake_case(created_by, is_public)
- ready-check가 이 컬렉션명 불일치를 BLOCKER로 잡아 FAIL 처리했고, 그 덕에 코드 짜기 전에 멈춰서 "그래서 진짜 SoT가 뭔데?"를 정면으로 마주했다.

✅ Keep (잘된 점)

- 문서를 꼼꼼히 써둔 덕분에 F-MEM 명세-에러코드-접근정책을 바로 끌어다 쓸 수 있었다.
- 배포된 firestore.rules를 단일 진실원천(SoT)으로 삼은 판단이 결국 맞았다.

실제로 동작을 강제하는 건 rules니까. references 모듈이 이미 같은 컨벤션이라 일관성도 챙겼다.

- 게이트가 막아준 덕에 잘못된 컬렉션명으로 코드를 쌓는 사고를 예방했다.

⚠️ Problem (아쉬운 점)

- 문서를 많이 만든 것 ≠ 문서가 일치하는 것. 산출물이 늘수록 같은 개념(회고록)의 컬렉션명/필드 표기가 문서마다 갈라졌고, 이게 구현 직전에야 드러났다.

2) 회고록 상세 조회

- 기능 설명: 카드 클릭 시 개별 회고 상세를 본다. 로그인해야 열람 가능.

- 동작 과정: 비로그인 상태로 카드 클릭 → 로그인 페이지로 리다이렉트 / 로그인 상태면 /memoirs/:id 상세 렌더.

[User Flow] 006 – 회고록 공유

HalleMalle

프로젝트참고주제회고록

로그아웃

< 회고록 목록으로

회고록 작성

제목

[TIL] 20260618 오늘의 교훈: 마감일을 지키는 것은 중요하다

39 / 100

내용

이번 프로젝트를 진행하며 "마감일을 지키는 것"이 단순한 약속이 아니라 팀 전체의 신뢰와 직결된다는 것을 절실히 느꼈다. KPT 형식으로 회고록을 남긴다.

★ Keep (잘한 점 · 유지할 것)

- 매주 시작 시 마일스톤을 잘게 쪼개 "이번 주에 끝낼 것"을 명확히 정의했다.
- 작업을 시작하기 전 테드라인을 역산해 일정을 세운 덕분에 막판에 몰리는 일이 줄었다.
- 진행 상황을 매일 짧게 공유해, 늦어지는 작업을 팀이 일찍 알아챌 수 있었다.

⚠ Problem (아쉬운 점 · 문제)

- 초반에 "이 정도면 금방 끝나겠지"라고 낙관하다가 예상보다 오래 걸려 마음을 한 번 넘겼다.
- 한 작업이 밀리자 그 뒤에 의존하던 작업들이 연쇄적으로 지연됐다.
- 마감에 임박해서야 공유하는 바람에, 팀이 함께 대응할 시간이 부족했다.

★ Try (다음에 시도할 것)

이번 회고록 작성 시 마감일(DDL)을 팀 전체에 공개하고, 지난 일

667 / 10000

작성 완료취소

3) 회고록 작성 F-MEM-002

- 기능 설명: 프로젝트 경험을 전체 공개 회고록으로 기록한다.

- 동작 과정:

a. /memoirs/write에서 제목·내용 입력 (관련 프로젝트 연결 선택 가능).

b. visibility=PUBLIC으로 설정, 작성자를 현재 사용자로 지정해 저장.

c. 필수값 누락 또는 관련 구인 부재 시 작성 차단.

[User Flow] 006 – 회고록 공유

HalleMalle

프로젝트참고주제회고록

로그아웃

< 회고록으로

회고록 수정

제목

[KPT] 20260618 오늘의 교훈: 마감일을 지키는 것은 중요하다

39 / 100

내용

이번 프로젝트를 진행하며 "마감일을 지키는 것"이 단순한 약속이 아니라 팀 전체의 신뢰와 직결된다는 것을 절실히 느꼈다. KPT 형식으로 회고를 남긴다.

★ Keep (잘한 점 · 유지할 것)

- 매주 시작 시 마일스톤을 잘게 쪼개 "이번 주에 끝낼 것"을 명확히 정의했다.
- 작업을 시작하기 전 테드라인을 역산해 일정을 세운 덕분에 막판에 몰리는 일이 줄었다.
- 진행 상황을 매일 짧게 공유해, 늦어지는 작업을 팀이 일찍 알아챌 수 있었다.

⚠ Problem (아쉬운 점 · 문제)

- 초반에 "이 정도면 금방 끝나겠지"라고 낙관하다가 예상보다 오래 걸려 마감을 한 번 넘겼다.
- 한 작업이 밀리자 그 뒤에 의존하던 작업들이 연쇄적으로 지연됐다.
- 마감에 임박해서야 공유하는 바람에, 팀이 함께 대응할 시간이 부족했다.

★ Try (다음에 시도할 것)

이번 주에 마감일을 지키기 위해 다음 주에 대해 계획을 세웠다. 이번 주에 마감일을 지키기 위해 다음 주에 대해 계획을 세웠다.

667 / 10000

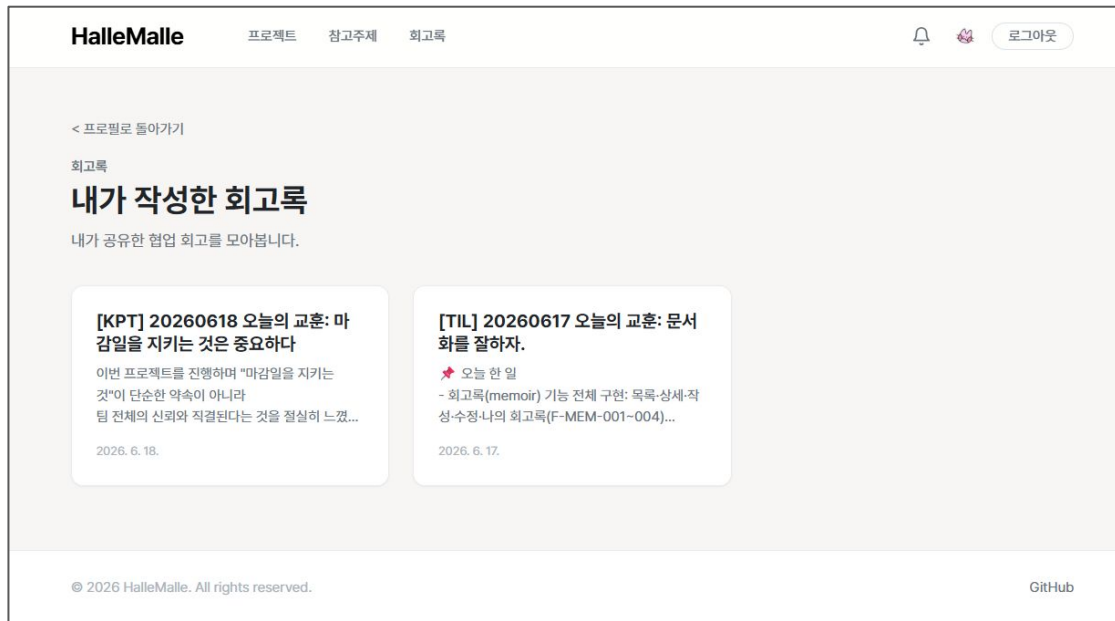
수정 완료취소

4) 회고록 수정 F-MEM-003

- 기능 설명: 작성자 본인이 자신의 회고록을 수정한다.

- 동작 과정: /memoirs/:id/edit → 작성자 검증(서버) 후 제목·내용 수정. 비작성자 차단.

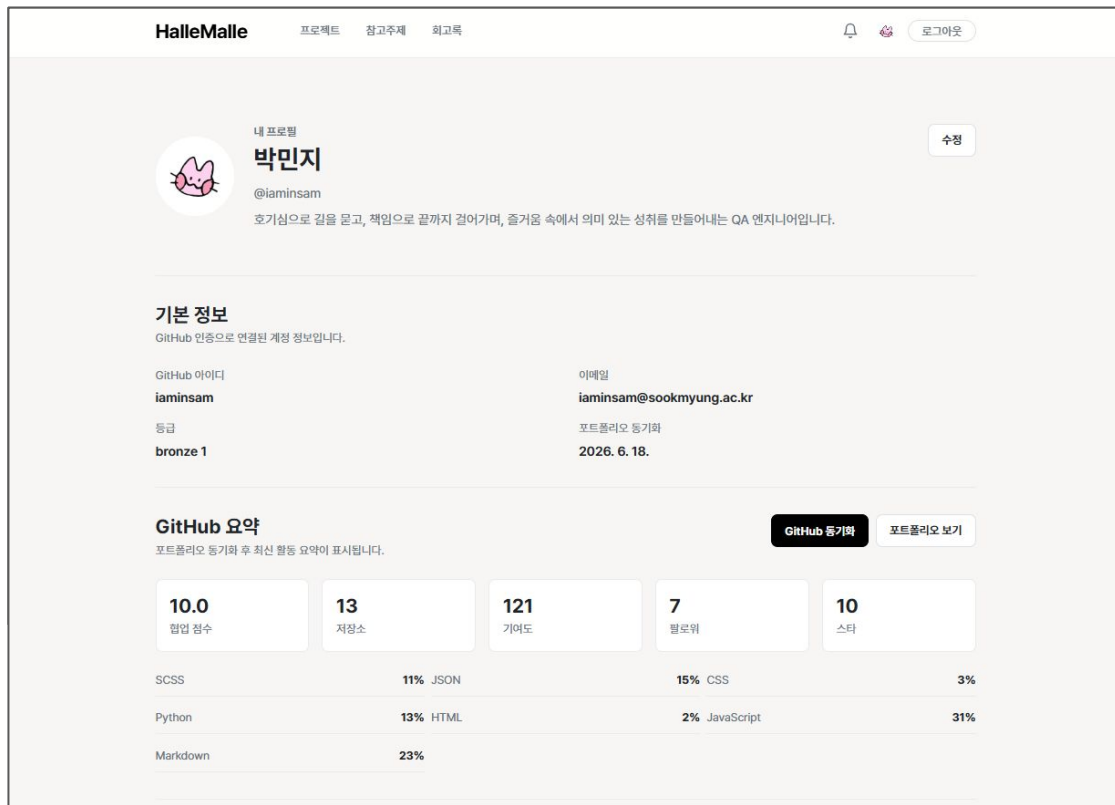
[User Flow] 006 – 회고록 공유



5) 나의 회고록 모아보기 F-MEM-004

- 기능 설명: 본인이 작성한 회고록만 모아 본다.
- 동작 과정: /profile/memoirs/my-article
→ 현재 사용자가 작성자인 회고록만 조회.

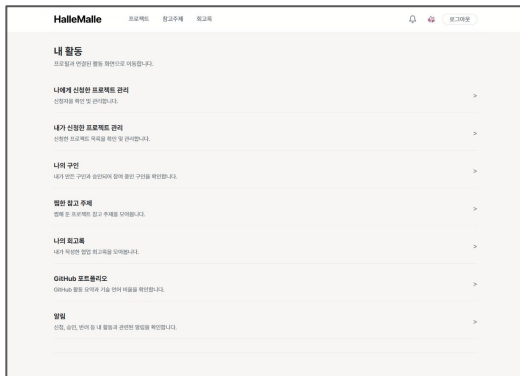
[User Flow] 007 – 내 프로필 & 포트폴리오



1) 내 프로필 조회 F-PROF-001

- 기능 설명: 프로필 이미지·아이디·노출 사용자명·GitHub 연결 상태를 확인하고, 각 모아보기로 이동하는 허브 역할.

- 동작 과정: /profile 진입 → 본인 프로필 정보 + GitHub 연결 상태 표시 → 내 구함글·신청·찜·회고록·알림 등으로 분기.



[User Flow] 007 – 내 프로필 & 포트폴리오

HalleMalle

프로젝트참고주제회고록

로그아웃

내 프로필

박민지

@iaminsam

호기심으로 길을 묻고, 책임으로 끝까지 걸어가며, 즐거움 속에서 의미 있는 성취를 만들어내는 QA 엔지니어입니다.

이름

박민지_2110669

이름은 2~20자, 공백 없이 입력해주세요.

소개

호기심으로 길을 묻고, 책임으로 끝까지 걸어가며, 즐거움 속에서 의미 있는 성취를 만들어내는 QA 엔지니어입니다. 단순히 "잘하고 싶다"는 마음만 있는 것이 아니라, 왜 하는지 알고, 스스로 납득한 방향으로 움직이며, 그 과정에서 몰입하고, 배우고, 책임지고, 결국 현실의 결과로 증명해내고자 노력하고 있습니다.

소개는 200자 이내로 입력할 수 있습니다.

저장

취소

2) 프로필 정보 수정 F-PROF-002 / 최초 상태 F-AUTH-003

- 기능 설명: GitHub 로그인 시 자동으로 채워진 기본 프로필을 사용자가 직접 다듬는다(이름·소개 수정).

- 동작 과정:

a. 최초 가입(첫 로그인) 시 자동 설정 — 별도 입력 없이 GitHub 계정 정보로 프로필이 생성된다.

- 프로필 이미지: GitHub 프로필 사진을 그대로 사용.

- 이름: GitHub 표시 이름(또는 GitHub 아이디)으로 초기화, 아래에 @GitHub아이디 표기

- 소개: GitHub bio가 있으면 가져오고, 없으면 빈 값 → 화면에 소개 문단이 아예 표시되지 않음.

- 협업 점수 10.00 · 티어 bronze 1로 시작, GitHub 지표(레포 · 기여 · 팔로워 · 스타)는 동기화 전이라 0으로 표시.

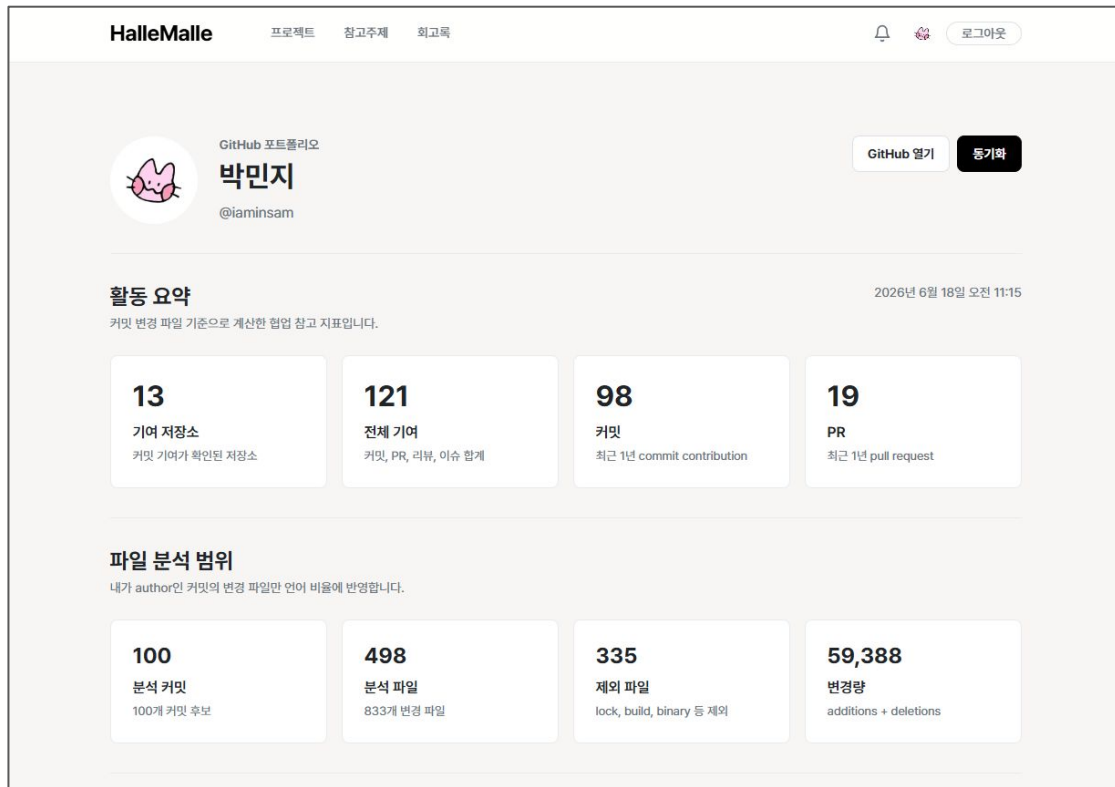
b. 사용자가 프로필 수정에 진입 → 이름·소개 입력란이 현재 값으로 채워진다(소개가 비어 있으면 "나를 소개하는 문장을 입력해주세요." 안내 문구(플레이스홀더)만 보임).

c. 입력값을 검증한다 — 이름 2~20자 · 공백 불가 · 중복 불가, 소개 200자 이내.

d. 검증 통과 시 저장 → 프로필 화면에 즉시 반영. 이때부터 소개를 입력했으면 소개 문단이 노출된다.

e. 형식 오류 · 중복 시 저장이 차단되고 안내 메시지를 표시.

[User Flow] 007 – 내 프로필 & 포트폴리오



GitHub 포트폴리오 조회 F-PORT-001

- 기능 설명: GitHub 기반 개인 포트폴리오 (레포·언어·기여도 요약)를 협업 판단 자료로 제공한다.

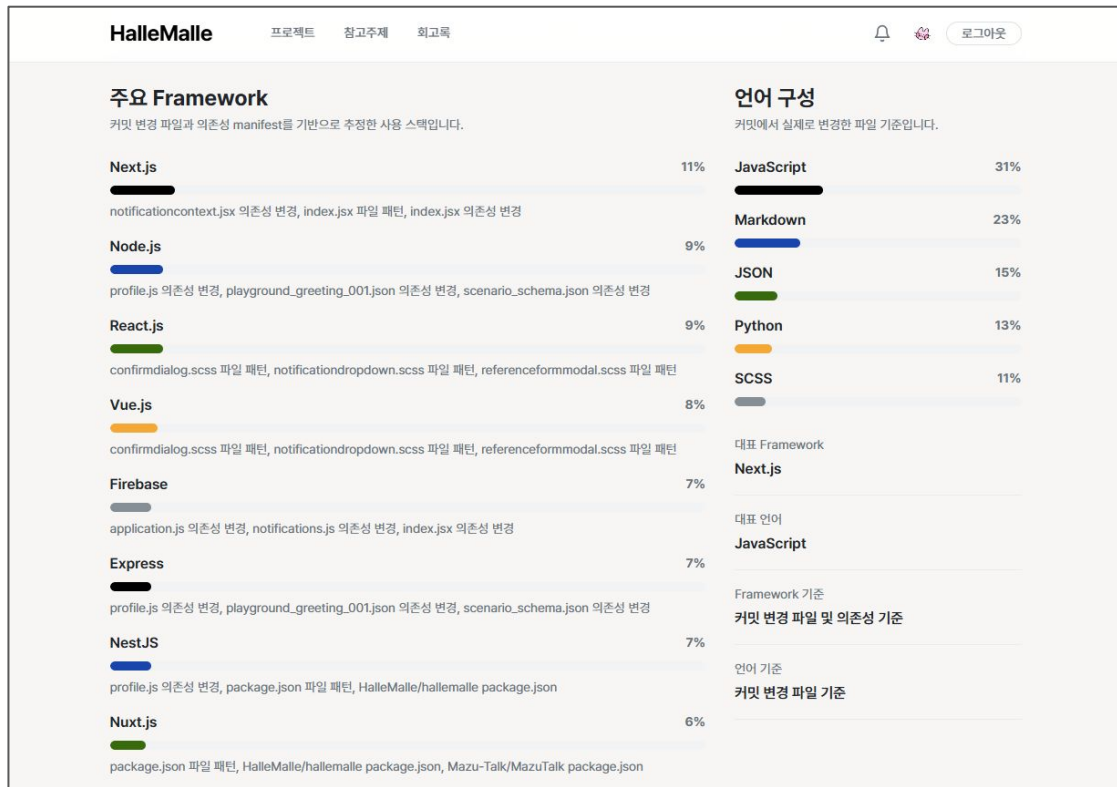
- 동작 과정:

a. /profile/portfolio 진입 → 저장된 포트폴리오 스냅샷 조회.

b. 스냅샷 없으면 syncRequired=true → 동기화 안내.

c. 있으면 기본 정보·레포지토리·언어 사용 비율·주요 프레임워크 분석 대시보드 표시.

[User Flow] 007 – 내 프로필 & 포트폴리오



GitHub 포트폴리오 조회 F-PORT-001

- 기능 설명: GitHub 기반 개인 포트폴리오 (레포·언어·기여도 요약)를 협업 판단 자료로 제공한다.

- 동작 과정:

a. /profile/portfolio 진입 → 저장된 포트폴리오 스냅샷 조회.

b. 스냅샷 없으면 syncRequired=true → 동기화 안내.

c. 있으면 기본 정보·레포지토리·언어 사용 비율·주요 프레임워크 분석 대시보드 표시.

[User Flow] 007 – 내 프로필 & 포트폴리오

HalleMalle			프로젝트	참고주제	회고록	🔔	🔒	로그아웃
기여 저장소			최근 1년 동안 커밋 기여가 확인된 저장소입니다.					
HalleMalle/hallemalle			Original · Public		22,082 changes			
locomoco-studio/OdiDuji			Original · Public		15,431 changes			
Mazu-Talk/MazuTalk			Original · Public		3,955 changes			
11mLLM/2026-Ollama			Original · Public		10,809 changes			
iaminsam/OneBite.JavaScriptChallenge			Original · Public		67 changes			
locomoco-studio/Odi-Duji			Original · Public		2,938 changes			
iaminsam/PyladiesGitTutorial			Original · Public		11 changes			
iaminsam/WebSystemDesign2026			Original · Public		139 changes			

GitHub 포트폴리오 조회 F-PORT-001

- 기능 설명: GitHub 기반 개인 포트폴리오 (레포·언어·기여도 요약)를 협업 판단 자료로 제공한다.

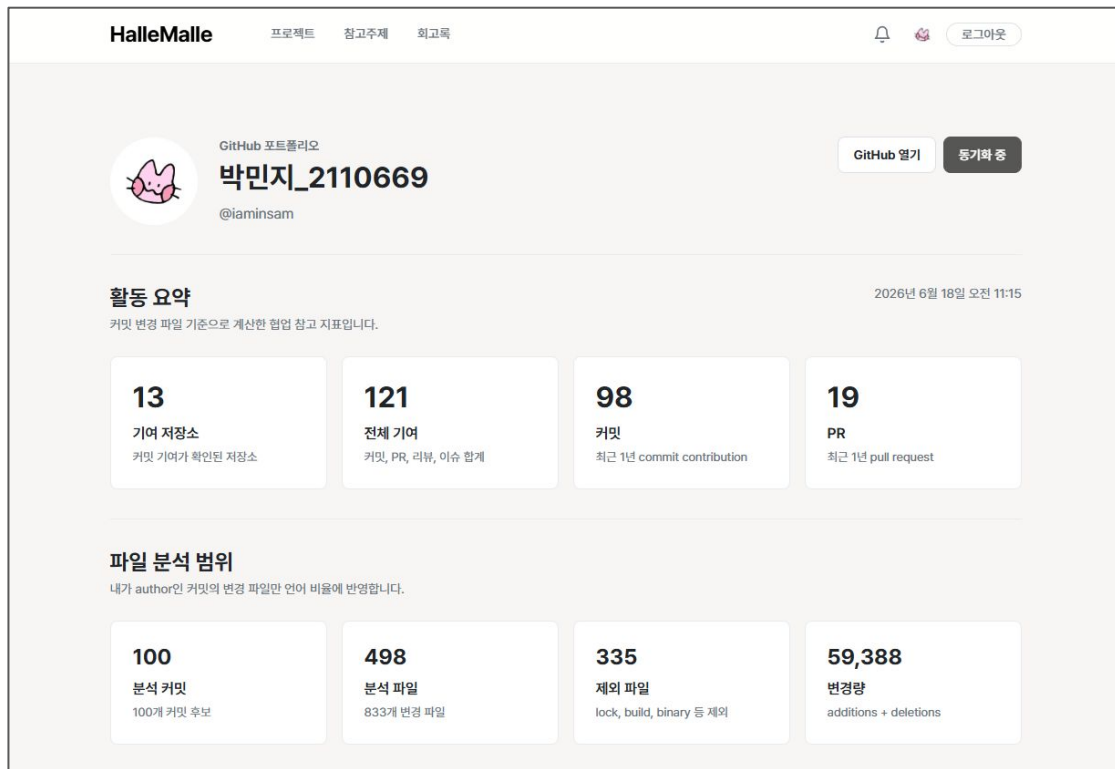
- 동작 과정:

a. /profile/portfolio 진입 → 저장된 포트폴리오 스냅샷 조회.

b. 스냅샷 없으면 syncRequired=true → 동기화 안내.

c. 있으면 기본 정보·레포지토리·언어 사용 비율·주요 프레임워크 분석 대시보드 표시.

[User Flow] 007 – 내 프로필 & 포트폴리오



4) GitHub 포트폴리오 동기화 F-PORT-002

- 기능 설명: GitHub에서 사용자·레포·언어·커밋(기여도) 정보를 수집·정규화해 스냅샷으로 저장한다.

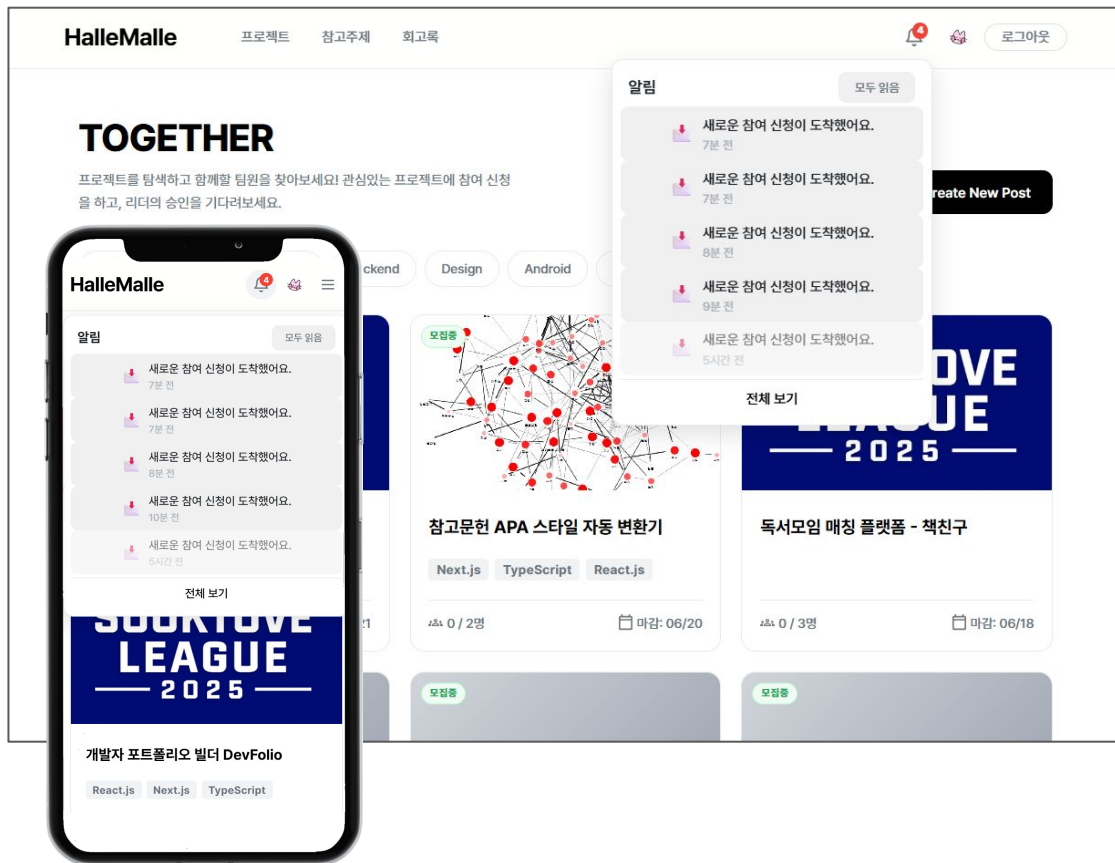
- 동작 과정:

a. 동기화 요청 → GitHub 기본 정보 → 레포 목록 → 레포별 언어 → 기여도 순으로 수집.

b. 서비스 포맷으로 정규화 후 스냅샷 생성/갱신.

c. 일부 수집 실패 시 실패 항목만 기록하고 나머지는 저장(외부 장애가 전체로 전파되지 않음). 요청 한도 초과 시 기존 스냅샷 유지.

[User Flow] 008 – 알림 & 공통 UX



1) 알림 요약(헤더 드롭다운) F-NOTI-001

- 기능 설명: 헤더 알림 버튼으로 최근 알림을 빠르게 확인하고, 미확인 개수를 뱃지로 알린다.

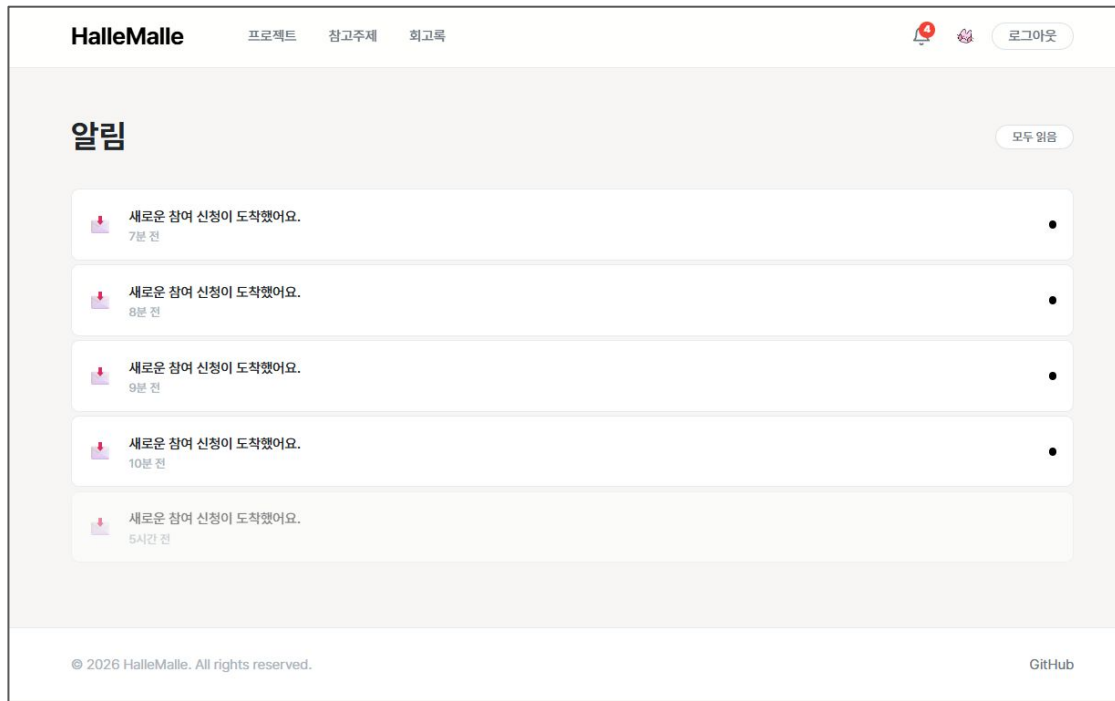
- 동작 과정:

a. 헤더 벨에 미확인 수 뱃지 표시.

b. 클릭 → 드롭다운에 최근 알림(기본 5건) 표시, 확인한 알림은 음영 처리.

c. "전체 보기"로 알림 페이지 이동.

[User Flow] 008 – 알림 & 공통 UX



2) 알림 목록(전체 페이지) F-NOTI-002

- 기능 설명: 본인이 받은 전체 알림을 최신순으로 확인한다.

- 동작 과정: /profile/notifications → 현재 사용자 소유 알림만 조회(타인 알림 미포함) → 항목 클릭 시 읽음 처리·관련 항목으로 이동.

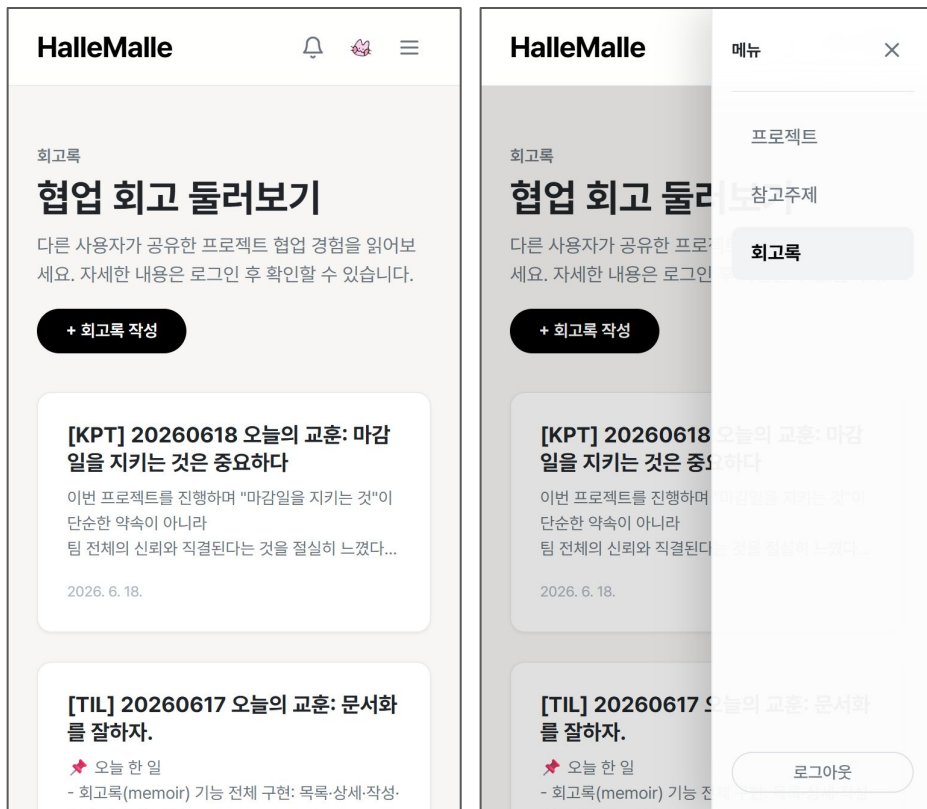
3) 알림 생성 트리거 (연계)

- 기능 설명: 신청·승인·반려 시점에 수신자에게 알림이 생성된다.

- 동작 과정: 신청 발생 → 구함글 작성자에게 알림 / 승인·반려 → 신청자에게 알림.

- 새로그침 기준 반영 — 실시간 푸시 아님.

[User Flow] 008 – 알림 & 공통 UX

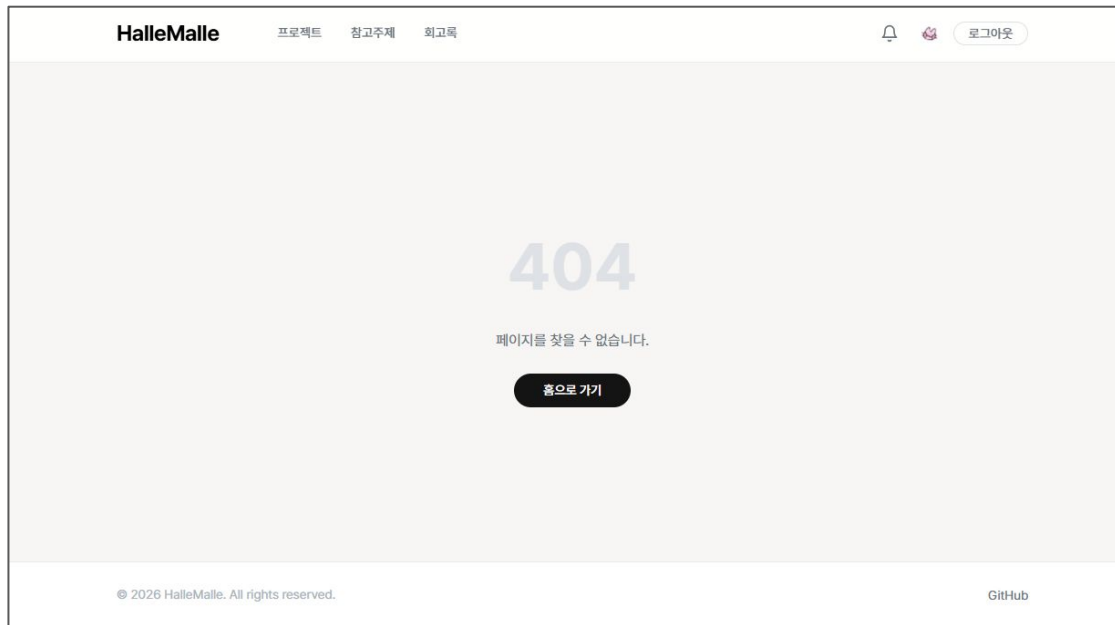


4) 모바일 글로벌 내비게이션 REQ-032 · NFR-010

- 기능 설명: 모바일에서 프로젝트·참고주제·회고록 GNB를 햄버거 메뉴로 제공하고, 로그인/로그아웃은 사이드드로어 최하단에 배치한다.

- 동작 과정: 모바일 진입 → 데스크톱 nav 숨김·햄버거 노출 → 탭하면 우측 드로어 오픈 (메뉴 3종 + 하단 로그인/로그아웃) → Esc·바깥 클릭·경로 이동 시 닫힘.

[User Flow] 008 – 알림 & 공통 UX



5) 미정의 경로(404) F-SYS-001

- 기능 설명: 등록되지 않은 경로 요청에 404 상태와 홈 복귀 경로를 제공한다.

- 동작 과정: 잘못된 URL 접근 → 404 화면 표시 → "홈으로 가기" 링크 제공.

설계 / 코드

폴더 구조

```
.
├─ src/
│   ├── api/           # Firebase 접근 계층 (도메인별 1파일)
│   │   ├── firebase.js # SDK 단일 진입점
│   │   ├── auth.js     # GitHub OAuth·계정 생성
│   │   ├── project.js  # 구함글(togethers) CRUD
│   │   ├── application.js # 참여 신청/승인/반려
│   │   ├── references.js # 참고주제 조회/정렬
│   │   ├── likes.js    # 참고주제 추천(Like)
│   │   ├── scraps.js   # 찜(스크랩)
│   │   ├── retrospectives.js # 회고록(memoirs) CRUD
│   │   ├── notifications.js # 알림 CRUD
│   │   └── profile.js  # 프로필·GitHub 포트폴리오 분석
│   ├── hooks/         # 비즈니스 로직 (useReferences, useRetrospectives, useProjects ...)
│   ├── pages/         # 라우트 단위 화면 (20+)
│   ├── components/    # 재사용 UI (layout · 도메인 카드/모달 · common)
│   ├── contexts/      # 전역 상태 (AuthContext · NotificationContext)
│   ├── styles/        # SCSS 토큰·믹스인 (variables / mixins / base)
│   ├── utils/         # 순수 헬퍼
│   └── router.jsx     # 라우트 단일 선언
```

폴더 구조

API (Firebase) Hooks (로직) Pages (화면)의 계층 강제 분리

→ 각 계층은 자기 바로 아래 계층만 호출하고, 위 계층을 모른다.

의존 방향은 항상 한쪽(pages→hooks→api→Firebase)이다.

계층	책임	하면 안 되는 것
pages/components	화면 렌더링, 사용자 입력 받기	Firebase·쿼리 직접 호출
hooks	상태(데이터·로딩·에러) 관리, API 호출 조합	JSX 렌더
api	Firestore 쿼리·뮤테이션·데이터 정규화	React 상태/화면 의존
api/firebase.js	Firebase SDK 초기화	(다른 api는 여기만 import)

폴더 구조

구체적인 예시 — 참고주제 화면 1개가 만들어지는 경로:

ReferenceList(page) → useReferences(hook) → references.js / like.js / scraps.js (api) → firebase.js → Firestore
화면만 상태·로딩·에러 관리 쿼리/뮤테이션·정규화 SDK 진입점

- 화면(ReferenceList)은 useReference()만 호출하고 Firestore를 전혀 모른다.
- 훅(useReferences)이 목록·추천·찜 API를 조합하고 로딩/에러 상태를 만든다.
- API(references.js 등)만 Firestore를 만지고, 그조차 firebase.js 단일 진입점을 거친다.

이렇게 나눈 이유

1. Firebase를 다른 BaaS로 바꿔도 api/ 만 수정
2. 같은 로직(훅)을 여러 화면에서 재사용
3. 화면·로직·동선을 따로 테스트·디버깅
4. 관심사가 섞이지 않아 쉬운 유지보수

주요 모듈

계층	모듈	역할
접근	api/firebase.js	FirebaseSDK 단일 진입점
접근	api/*.js	도메인별 Firestore CRUD
로직	hooks/use*.js	조회·뮤테이션·로딩/에러 상태 캡슐화
전역	contexts/AuthContext·NotificationContext	로그인 세션·알림 미확인 수 공유
보호	components/layout/ProtectedRoute.jsx	비로그인 시 /login 리다이렉트
UI	components/(ProjectCard·ReferenceFormModal·NotificationDropdown·Header...)	도메인 카드/모달/헤더

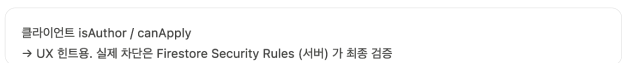
데이터 흐름



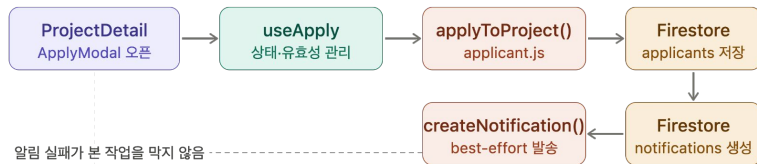
전역 공유 Context



보안 모델



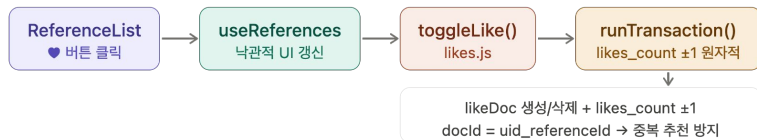
Flow A — 팀원 신청 · 승인 · 알림



오너 측



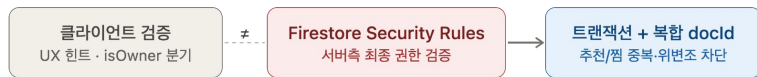
Flow B — 참고주제 추천 (트랜잭션)



Flow C — 알림 전역 상태 (Context)



보안 레이어

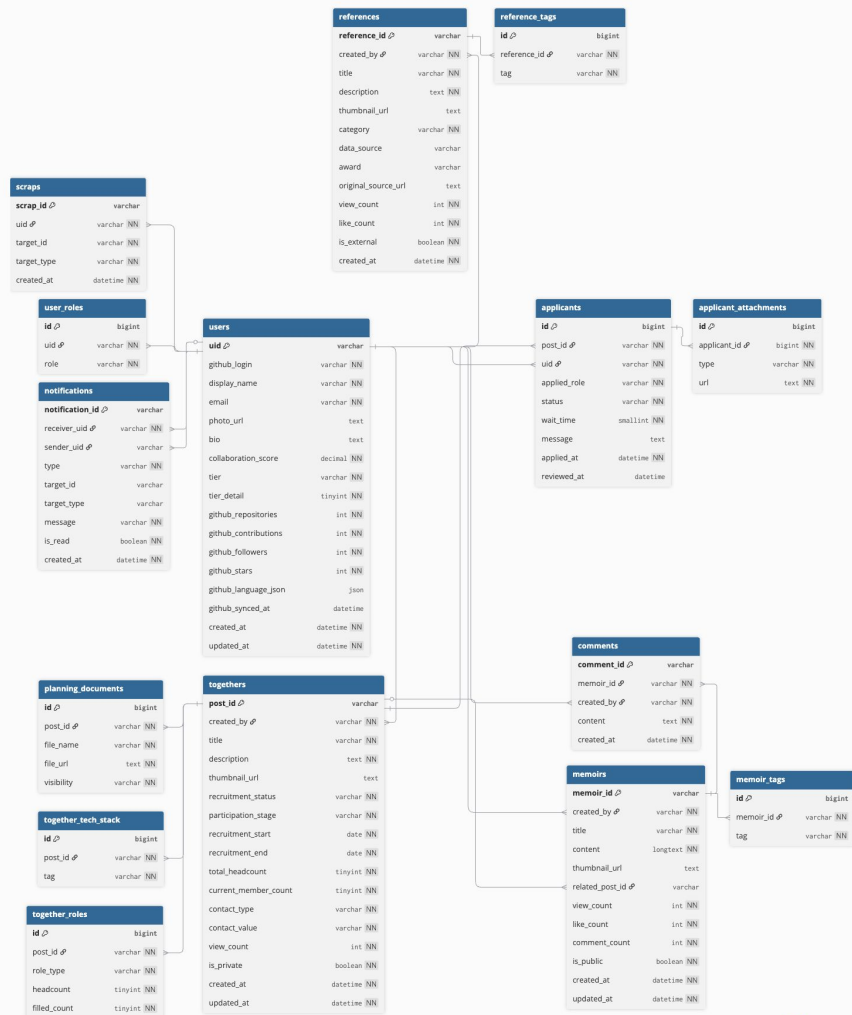


■ Pages
 ■ Hooks
 ■ API
 ■ Firestore
 ■ Security Rules
 ■ 트랜잭션

DB 구조 및 데이터 저장 구조

테이블별 간략한 설명 (각 설명별 '~모음' 또는 '~테이블' 생략)

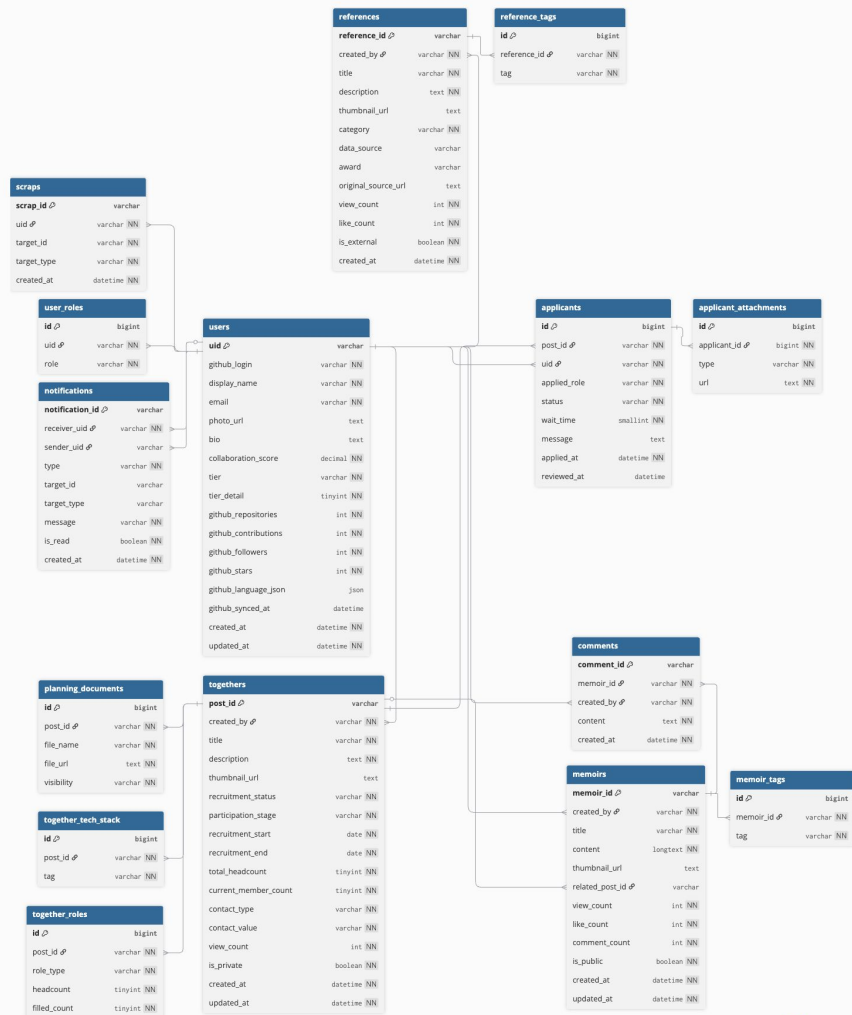
post_id	프로젝트 id
file_name	업로드된 첨부 파일의 원본 파일명
file_url	Firestore 용량 제한(1MB 이하)을 준수하여 Base64 문자열로 인코딩된 파일 데이터
visibility	문서의 열람 권한 범위 지표 (public: 전체 공개 / approved_only: 승인된 팀원만 조회 가능)



DB 구조 및 데이터 저장 구조

테이블별 간략한 설명 (각 설명별 ‘~모음’ 또는 ‘~테이블’ 생략)

applicants	유저가 특정 프로젝트에 지원한 내역
applicant_attachments	지원자가 첨부한 첨부파일 (포트폴리오 등)
references	레퍼런스 아카이브 (시스템 등록 혹은 유저 등록)
reference_tags	레퍼런스 게시물 분류를 위한 태그



테이블별 간략한 설명 (각 설명별 ‘~모음’ 또는 ‘~테이블’ 생략)

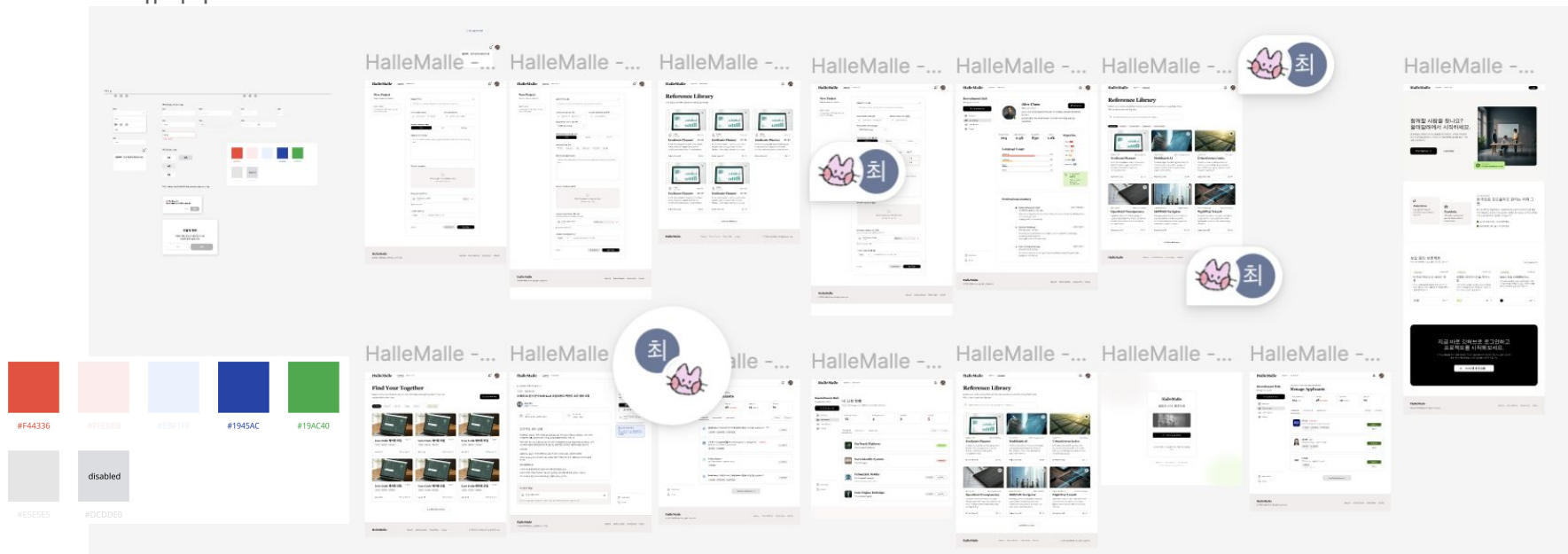
notifications	프로젝트 종료 후 작성하는 회고록
---------------	--------------------



UI/UX 설계 - 디자인

UI/UX 디자인 원칙 및 시스템화

- 프로젝트의 성격에 맞춰, 신뢰감을 주는 다크 네이비, 딥 그린, 클린 화이트를 메인 컬러 스펙트럼으로 채택하여 시각적 피로도를 낮춤
- primary, disabled, success, warning, danger, info 등의 색상 설계 및 통일
- xs, sm, base, lg, xl, 2xl, 3xl 또는 normal, medium 등을 활용하여 font-size, font-weight, line-height, shadow, spacing, radius 규격화



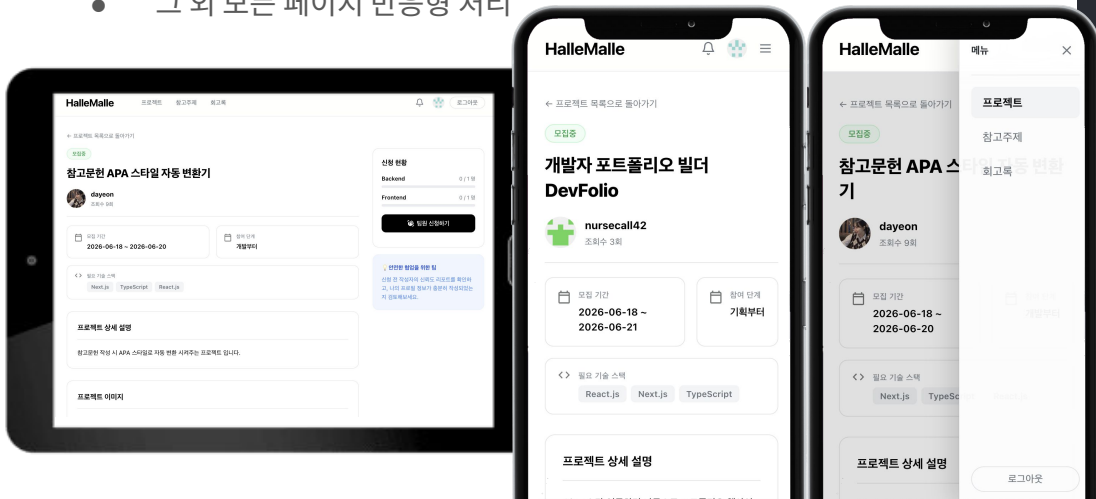
UI/UX 설계 - 반응형

반응형

- SCSS의 특성을 활용해 반응형 뷰포트를 mixin으로 규격화 및 관리
- 반응형 뷰포트 그리드 시스템 도입

적용 예시

- 상단 헤더바 모바일 환경에서는 햄버거 버튼과 사이드 모달로 처리
- 그 외 모든 페이지 반응형 처리



src > styles > mixins > _media.scss > ...

You, 2 weeks ago | 1 author (You)

@use "@/styles/variables" as *;

```
1
2
3 @mixin mobile {
4   @media (max-width: #{breakpoint-sm - 1px}) {
5     @content;
6   }
7 }
```

```
8
9 @mixin tablet {
10  @media (min-width: breakpoint-sm) and (max-width: breakpoint-md) {
11    @content;
12  }
13 }
```

```
14
15 @mixin desktop {
16  @media (min-width: breakpoint-lg) {
17    @content;
18  }
19 }
```

```
20
21 @mixin tablet-up {
22  @media (min-width: breakpoint-sm) {
23    @content;
24  }
25 }
```

```
26
27 @mixin respond-to($bp) {
28  @media (max-width: $bp) {
29    @content;
30  }
31 }
```

UI/UX 설계 - 편의성

예외 처리 UI

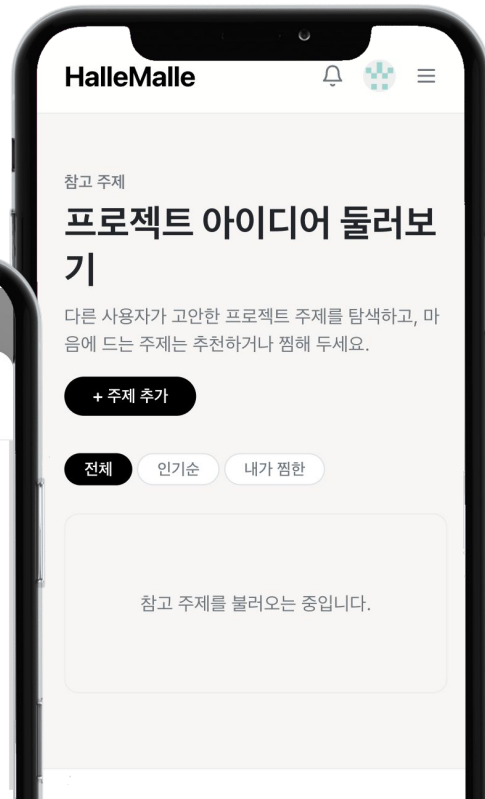
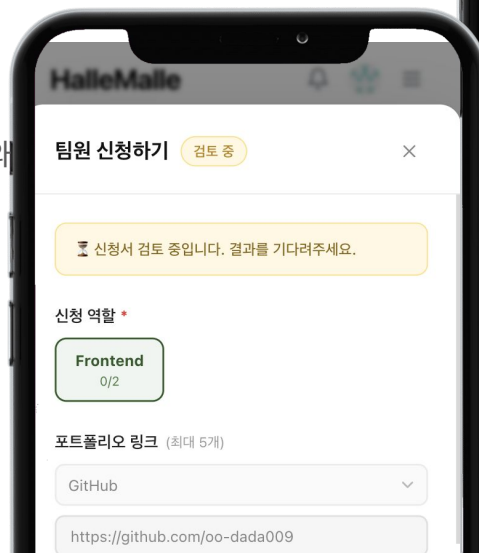
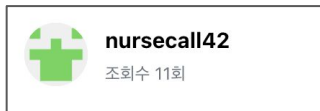
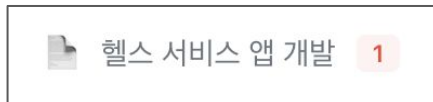
- 빈 화면이 노출될 수 있는 상황(검색 결과 없음, 삭제된 프로젝트 접근 등)을 대비해 플레이스홀더 및 안전한 안내 팝업 가드를 배치하여 이탈률 방지.

직관적인 모달 워크플로우

- 팀원 신청 시 상세 페이지 이탈 없이 그 자리에서 즉시 지원 역할을 선택하고 포트폴리오를 제출할 수 있는 원-클릭 지원 모달 인터페이스 구현

실시간 정보 및 시각적 피드백

- 모집글 카드의 우측 상단에 실시간 조회수(view_count)와 상태와 실시간 알림 피드백 인프라를 UI 상단 noti피케이션 모달과 연동



기능별 핵심 코드 - 게시물 생성

프론트엔드: 폼 데이터 구조화 및 생성 요청

(ProjectForm & ProjectCreate)

- 사용자가 폼에 입력한 다형성 데이터(텍스트, 가변 배열, 첨부파일 등)를 파이어베이스 백엔드 스키마에 맞춰 안전하게 정제(Sanitize), 비동기 호출을 통해 생성을 요청함.
- stage와 participationStage를 동시에 명시하여 수신부(project.js)에서 어떤 키로 구조분해를 하더라도 undefined 에러가 발생하지 않도록 방어
- 비동기 처리가 완료되면 데이터베이스에서 리턴받은 고유 newPostId를 활용하여 사용자를 부드럽게 상세 페이지로 전환

```
src > components > ProjectForm > index.jsx > ProjectForm
85   export default function ProjectForm({
242     const handleSubmit = async (e) => {

257     const data = {
258       title: title.trim(),
259       description: description.trim(),
260       startDate: startDate || null,
261       endDate: endDate || null,
262       headcount: totalSlots,
263       status,
264       // 수신부(project.js 또는 부모 컴포넌트)에서 'stage'나 'participationStage'
265       // 어떤 것을 구조분해하더라도 undefined가 되지 않도록 두 키 모두에 값을 명시합니다.
266       stage: stage,
267       participationStage: stage,
268       positions,
269       contactType,
270       contactValue: contactValue.trim(),
271       thumbnail: thumbnail || null,
272       attachments: hasAttachmentStage
273       ? planningDocs.map(({ name, file, url, visibility }) => ({
274         name,
275         file: file || null,
276         url: url || null,
277         visibility,
278       }))
279       : [],
280       techStack: Array.from(selectedStack),
281     };
282
283     await
284   } catch
285   con

src > pages > ProjectCreate > index.jsx > ProjectCreate > handleCancel
11   export default function ProjectCreate() {
17     const handleSubmit = async (data) => {
21       try {
22         const newPostId = await createProject(data, user?.uid);
23
24         if (newPostId) {
25           navigate(`/togethers/${newPostId}`);
26         }
27       } catch (err) {
```

기능별 핵심 코드 - 승인 / 반려

프론트엔드: 상태별 조건부 렌더링 및 UI 바인딩 (ResponseList)

- 현재 활성화된 탭의 상태(pending, approved, rejected)에 따라 데이터가 비어 있을 경우 플레이스홀더 문구를 조건부 렌더링
- 데이터가 존재할 경우, 정렬된 지원자 리스트(sortedActiveList)를 .map() 함수로 순회하며 개별 지원자 카드 컴포넌트(<ApplicantCard />)로 동적 마운트

```
src > pages > ResponseList > index.jsx > ...
199   export default function ResponseList() {
523     <div className="responses-page__applicant-list">
524       {sortedActiveList.length === 0 ? (
525         <div className="responses-page__empty">
526           {activeTab === "pending" && "아직 대기 중인 신청자가 없습니다."}
527           {activeTab === "approved" && "승인된 신청자가 없습니다."}
528           {activeTab === "rejected" && "반려된 신청자가 없습니다."}
529         </div>
530       ) : (
531         sortedActiveList.map((applicant) => (
532           <ApplicantCard
533             key={applicant.id}
534             applicant={applicant}
535             onApprove={handleApprove}
536             onReject={handleReject}
537             isActionLoading={actionLoading === applicant.id}
538           />
539         ))
540       )}
```

백엔드: 상태 업데이트 및 실시간 알림 발송 API (approveApplication)

- 전달받은 지원서 고유 ID(applicantId)를 식별자로 삼아 applicants 컬렉션의 status 필드를 approved로 즉시 업데이트.
- **트리거 기반 알림 연동**
상태 변경이 완료되면 독립적인 비동기 함수 notifyApplicant를 연속 호출하여, 해당 지원자에게 APPLICATION_APPROVED 타입의 실시간 푸시 알림("참여 신청이 승인되었습니다.")을 즉각 적재

```
src > api > JS application.js > cancelApplication
205   export async function approveApplication(applicantId) {
206     await updateDoc(doc(db, APPLICANTS_COL, applicantId), {
207       status: "approved",
208       reviewed_at: serverTimestamp(),
209     });
210
211     await notifyApplicant(
212       applicantId,
213       "APPLICATION_APPROVED",
214       "참여 신청이 승인되었습니다."
215     );
216   }
```


기능별 핵심 코드 - 포트폴리오

인증: GitHub 액세스 토큰 발급 (getGithubAccessToken)

- Firebase 재인증 팝업(reauthenticateWithPopup)으로 read:user·repo 스코프의 GitHub 액세스 토큰을 발급받음.
- 발급한 토큰을 이후 모든 API 요청의 인증 헤더(Authorization: Bearer)에 실어 호출.

데이터 페칭: 용도별 엔드포인트 분리

- 프로필·레포지토리·기여 정보는 GraphQL 엔드포인트 (fetchGithubGraphql, POST)로 한 번에 조회.
- 커밋 상세(파일별 변경 내역)는 REST 엔드포인트(fetchGithubRest)로 조회하며, X-GitHub-API-Version 헤더로 API 버전을 고정.

응답 상태 기반 예외 처리

- 응답 코드 403은 요청 한도 초과(PORT-002-RATE-LIMITED), 그 외 실패 응답은 PORT-002-PROVIDER-UNAVAILABLE로 분류해 사용자용 메시지로 변환.
- GraphQL 응답 본문에 errors가 포함되면 PORT-002-GRAPHQL-FAILED로 처리해, HTTP는 성공했지만 쿼리가 실패한 경우까지 잡아냄.

수집 흐름

- 토큰 발급 → GraphQL로 레포/기여 조회 → 레포별 REST로 커밋 상세(파일 변경) 조회 → G-2 언어/프레임워크 분석의 입력으로 전달.

```
// 1) GitHub 액세스 토큰 발급 - Firebase 재인증 팝업 + 필요한 스코프 요청
async function getGithubAccessToken() {
  const provider = new GithubAuthProvider();
  provider.addScope("read:user");
  provider.addScope("repo");
  const result = await reauthenticateWithPopup(auth.currentUser, provider);
  return GithubAuthProvider.credentialFromResult(result).accessToken;
}
```

```
// 2) GraphQL 호출 (프로필/레포/기여) - Bearer 토큰
async function fetchGithubGraphql(query, variables, accessToken) {
  const response = await fetch(GITHUB_GRAPHQL_URL, {
    method: "POST",
    headers: { Authorization: `Bearer ${accessToken}`, "Content-Type": "application/json" },
    body: JSON.stringify({ query, variables })
  });
  if (response.status === 403)
    throw createProfileError("PORT-002-RATE-LIMITED", "GitHub 요청 한도를 초과했습니다.");
  if (!response.ok)
    throw createProfileError("PORT-002-PROVIDER-UNAVAILABLE", "GitHub 인증이 실패했습니다.");
  const payload = await response.json();
  if (payload.errors?.length) throw createProfileError("PORT-002-GRAPHQL-FAILED", "GraphQL 쿼리 실패");
  return payload.data;
}
```

```
// 3) REST 호출 (커밋 상세) - API 버전 헤더 포함
async function fetchGithubRest(path, accessToken) {
  const response = await fetch(createGithubApiUrl(path), {
    headers: {
      Authorization: `Bearer ${accessToken}`,
      Accept: "application/vnd.github+json",
      "X-GitHub-API-Version": "2022-11-28",
    },
  });
  if (response.status === 403) throw createProfileError("PORT-002-RATE-LIMITED", "GitHub 요청 한도를 초과했습니다.");
  return response.json();
}
```


어려웠던 점 / 해결 과정

[민지] 추천·찜의 낙관적 업데이트·롤백 + 추천 수 무결성 보장

어려웠던 점 (Problem)

- 추천(Like)·찜(Scrap) 버튼을 누를 때마다 서버 응답을 기다리면 클릭 반응이 느려 답답했고, 그렇다고 화면을 먼저 바꾸면 서버 요청이 실패했을 때 화면이 실제와 다른 "거짓 상태"로 남는 문제가 있었음. 특히 추천 → 즉시 취소를 빠르게 반복하는 동시성 상황에서 추천 수(likers_count)가 서버 값과 어긋나거나 음수로 갈 위험이 있었고, 추천 수는 비작성자도 변경할 수 있어야 하는 값이라 그대로 두면 카운트 위조까지 가능했음. 또한 주제를 삭제할 때 연관 문서(태그·추천)를 함께 정리해야 하는데, 삭제 순서에 따라 작성자 검증(Rules)이 실패할 수 있었음.

해결 과정 (Solution)

- 추천/찜 상태를 src/hooks/useReferences.js에서 관리하며, 버튼 클릭 시 서버 응답을 기다리지 않고 UI 상태(likedIds/scrappedIds)와 추천 수(likeCounts)를 먼저 갱신하는 낙관적 업데이트를 적용함. 서버 응답이 오면 응답값(result.isLiked, result.likesCount)으로 최종 동기화하고, 요청이 실패하면 직전 상태로 되돌리는 롤백 로직을 둬. 추천 수는 클라이언트가 누적하지 않고 매번 서버 응답값으로 덮어서, 빠른 연속 클릭 상황에서도 최종값이 항상 서버 기준으로 수렴하게 했고 Math.max(0, ...)로 음수를 차단함. 추천 수 무결성은 Firestore Rules에서 보장하도록 설계해, 추천 문서는 reference_likes/{uid}_{referenceId} 복합키로 사용자-주제당 1건만 존재하게 하고, references의 likes_count는 같은 커밋에서 본인 추천 문서가 생성/삭제될 때만 ± 1 변경을 허용함.
- 참고 주제 삭제(src/api/references.js의 deleteReference)는 주제 문서가 아직 존재하는 동안 연관 태그(reference_tags)·추천(reference_likes)을 같은 배치(batch)로 삭제하도록 순서를 잡았는데, Rules의 get/exists가 커밋 전 상태를 읽기 때문에 이 순서여야 연관 문서 삭제 시 작성자 검증이 통과함. 태그 동기화(syncTags)는 본문 저장과 분리해 기존 태그를 삭제한 뒤 신규 태그를 다시 추가하는 방식이며, 보조 작업이라 실패해도 본문 변경은 되돌리지 않는 best-effort로 둬.

[민지] GitHub 포트폴리오 – 언어 비율 / 프레임워크 추론

어려웠던 점 (Problem)

- GitHub의 기본 언어 통계는 레포 전체 바이트를 기준으로 하기 때문에, 포크·자동 생성 파일·설정 파일이 비율을 부풀려 "실제로 작성한 양"을 반영하지 못했음. 또한 GitHub은 어떤 프레임워크를 썼는지는 알려주지 않아 직접 추론해야 했는데, 의존성에 react-native 같은 토큰만 들어 있어도 React Native 개발자로 잡히는 등 단서 하나로 판단하면 오탐이 잦았고, lock 파일·빌드 산출물 같은 잡음 파일이 언어 집계를 흐리는 문제가 있었음

해결 과정 (Solution)

- src/api/profile.js에서 레포 통계 대신 각 커밋의 파일별 변경량(additions + deletions)을 언어별로 합산(aggregateCommitFileLanguages)해, "보유한 언어"가 아니라 "실제로 작성한 언어"의 비율을 산출하도록 설계함. 이때 확장자→언어 매핑이 되지 않거나 설정·빌드 산출물로 분류되는 파일은 classifyFileLanguage·shouldExcludeCommitFile로 집계에서 제외해 비율 왜곡을 방지함.
- 프레임워크는 그 프레임워크에서만 나오는 파일 경로 패턴과 변경된 코드 안의 패치 토큰이라는 두 가지 신호에 가중치를 줘 합산하고 점수가 높은 것부터 채택(FRAMEWORK_DETECTORS)했으며, React Native처럼 오탐이 잦은 경우에는 requiresStrongPathMatch 플래그로 실제 전용 파일(metro.config, .ios/.android 파일 등)이 있을 때만 인정하도록 감지 조건을 보수화함. 마지막으로 분석한 커밋·파일 수와 제외한 파일 수(stats)를 함께 기록해 분석 커버리지를 투명하게 남김.

[다연] 하위 컬렉션 무결성 및 연쇄 삭제(Cascade) 보장

어려웠던 점 (Problem)

- 메인 프로젝트 구인글(togethers)을 삭제할 때, 이와 연결된 서브 데이터(모집 역할군, 기술 스택, 기획 문서)는 물론, 해당 글에 지원했던 지원서(applicants), 포트폴리오 첨부파일 데이터(applicant_attachments)가 삭제되지 않고, DB 내 유령 데이터로 남아 메모리 및 데이터 정합성을 깨뜨리는 문제가 발생할 위험이 있었음.

해결 과정 (Solution)

- 프로젝트 삭제 로직(deleteProject)에서 메인 문서를 지우기 전, query와 where 문을 활용해 관련된 모든 하위 서브 컬렉션 스냅샷을 선행 조회하도록 아키텍처를 설계함.
- 루프 내에서 지원서 ID와 묶인 첨부파일 데이터를 역추적하여 일괄 수집한 뒤, JavaScript의 Promise.all 비동기 병렬 처리 기술을 도입하여 [서브 컬렉션 - 지원자 첨부파일 - 지원서 메인 문서 - 구인글 원본] 순으로 연쇄 삭제(deleteDoc) 처리. 데이터 무결성을 달성함.

협업 / 과정

역할 분담

도메인(기능) 단위로 분담

- 화면/API/문서를 한 사람이 세로로 책임지는 구조

분담 기준

- 페이지/도메인이 독립적이라 병렬 작업 시 충돌이 적도록 경계를 나눔

최다연	
페이지명	페이지 경로
홈 화면 (랜딩페이지)	/
구인 리스트 페이지	/togethers
구인 게시물 생성 페이지	/togethers/write
구인 게시물 상세 페이지	/togethers/:postId
구인 게시물 수정 페이지	
신청 관리 페이지 (나에게 신청)	/profile/responses
신청 관리 페이지 (내가 신청)	/profile/requests

박민지	
페이지명	페이지 경로
404 페이지	
로그인 페이지	/login
주제 참고 리스트 페이지	/references
회고록 리스트 페이지	/memoirs
프로필 페이지	/profile
내가 쓴 회고록 리스트 페이지	
나의 알림 페이지 (풀페이지)	/profile/notifications
포트폴리오 페이지 (Github)	/profile/portfolio/github

협업 도구

용도	도구
코드/버전관리	GitHub (https://github.com/HalleMalle/hallemalle)
코드 리뷰	GitHub Pull Request w/ PR Template.md
알림/소통	Discord (PR Webhook 연동 - PR 생성 및 머지 시 자동 알림)
백엔드/인프라	Firebase (Auth·Firestore·Storage) 공유 프로젝트

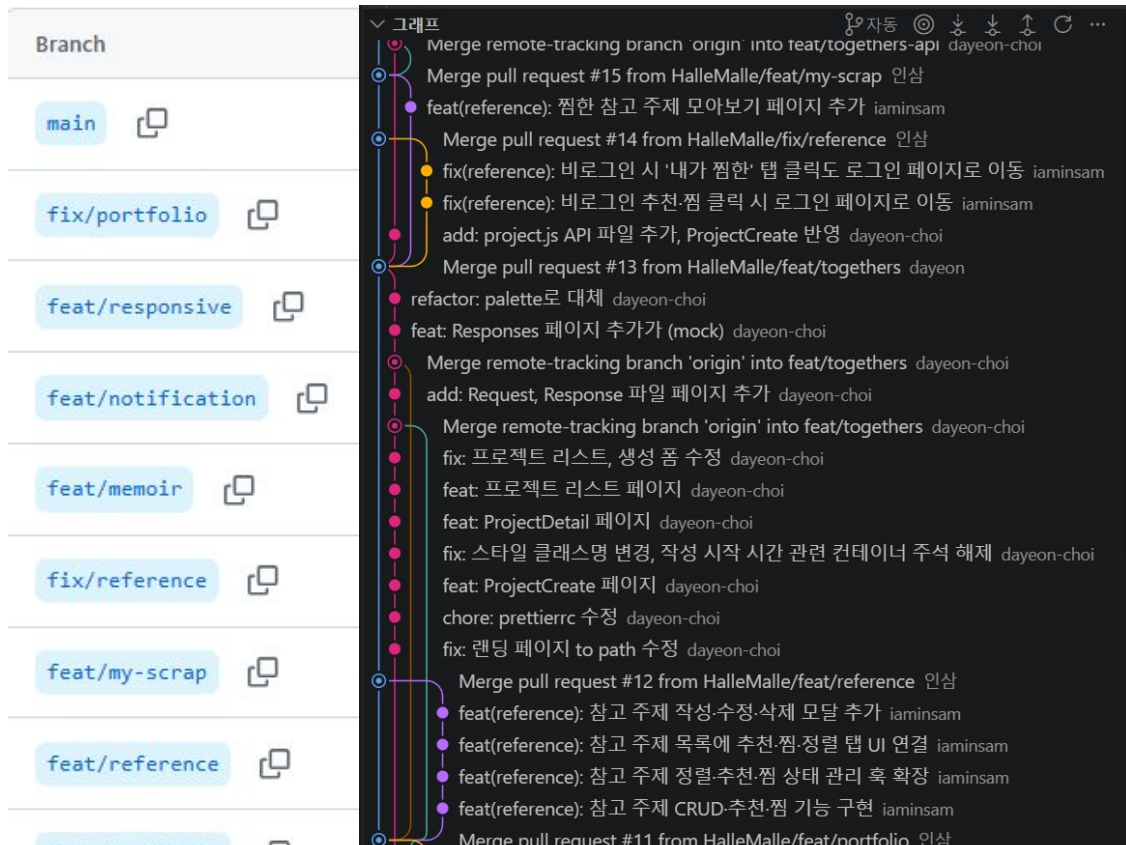
코드 리뷰

The screenshot displays two GitHub pull request reviews. The left review, by dayeon-choi, focuses on the login page, suggesting updates to the login note and service status. The right review, by iaminsam, points out an outdated code snippet in the project detail page and suggests displaying the creator's photo using the current user's photo URL.

알림/소통

The screenshot shows a Discord message from iaminsam. It contains a GitHub pull request update for the 'HalleMalle/hallemalle' repository, mentioning a pull request opened and closed. Below the update, there is a list of tasks and a note about the application API.

Git WorkFlow (브랜치 전략 및 커밋 메세지 컨벤션)



Branch Convention

기능별 브랜치 → PR → 리뷰 → main 병합
PR 단위는 한 기능 = 한 PR로 작게 유지
main에 직접 push하는 것을 금지함

Commit Message 태그(영어): 제목 (한국어)

feat : 새로운 기능을 추가
fix : 버그 수정
refactor : 프로덕션 코드 리팩토링
docs : 문서 수정
chore : 빌드 업무 수정, 패키지 매니저
수정, 패키지 관리자 구성 등 업데이트

개발 진행 흐름 (실제 PR 타임라인)

공통 기반 / 인증	#1 common · #5 login · #6 GitHub OAuth · #7 styles
프로필 / 포트폴리오	#9 user schema · #10 profile · #11 portfolio
구함 · 신청	#13 togethers · #16 togethers api · #17 신청 · #19 신청 관리
참고주제 / 회고 / 알림	#12 reference · #15 my-scrap · #18 회고록 · #20 알림
마무리	#21 반응형

의존성 관계를 고려하여,

기반(인증·스타일) → 핵심 도메인(구함 · 신청) → 부가 도메인(참고주제 · 회고 · 알림) → 반응형 마감 순으로 진행

주간 스크럼 및 문서화 관리

우리팀플할래		
프로젝트 일정		
Aa 이름	📅 날짜	🏷 태그
스크럼 5회차		
기획서 최종마감일	2026년 5월 24일	
[zoom] 제안서 주제 보고	2026년 5월 6일 오전 11	
기획서 마감일	2026년 5월 6일	
스크럼 4회차	2026년 5월 5일	
스크럼 3회차	2026년 4월 28일	
중간고사	2026년 4월 21일 → 2026년 4월 2	
스크럼 2차 (프로젝트 주제 선정)	2026년 4월 14일	
스크럼 1차 (팀 컨벤션)	2026년 4월 7일	
14주차 보고	2026년 6월 6일	
13주차 보고	2026년 5월 30일	
TODO	2026년 5월 20일 → 2026년 5월 2	
TODO	2026년 5월 13일 → 2026년 5월 1	
스크럼 8회	2026년 6월 2일	
스크럼 7회	2026년 5월 26일	
스크럼 6회	2026년 5월 21일	
스크럼 5회 (디자인 생성 / 일정 점검)	2026년 5월 12일	
최종제출마감	2026년 6월 18일	
+ 새 페이지		

팀 컨벤션

▼ 일정관리 / 출석 규칙

- 주간 스크럼 일정 (요일, 시간, 장소) : 화요일 17:00~19:00
- 데일리 스크럼 여부 및 방식 : 프로젝트 마감 2주전
- 회의 지각/결석 기준
 - 지각 시 페널티(벌칙)
 - 다음 강의시간에 커피를 사겠습니다
- 일정 취소 가능 기한 (약속 며칠 전까지 파토낼 수 있는지)
 - 24시간 전 (하루 전)
- 프로젝트 마감일 + 내부 마감일 (버퍼 포함)
 - 기획 문서 제출 : ~ 5/6
 - 데일리 스크럼 시작일 : ~5/26
 - 프로젝트 내부 마감일 : ~ 6/2
 - 프로젝트 최종 제출 : ~6/10
- 작업 지연 시 보고 기준
 - 막히는거 있으면 2시간 고민해보고 들고옵시다 🙏
- 블로커 발생 시 행동
 - 2시간 노력해보고 즉시 공유
- 스크립 조정 기준
 - 5월 19일쯤 (2주 진행) 해보고 구현 범위 축소

▼ 커뮤니케이션 규칙

- 응답 SLA
 - ex) 평일 24시간 내 응답
 - 남에는 최소 2시간 내 응답 / 밤 2:00~7:00

주 1회 주간 스크럼

목적: 진행 상황을 동기화 · 작업 재분배 · 블로커 (막힌 점) 공유 및 해소

진행 방식: 각자 아래 3가지를 공유

1. 지난 주에 한 일
2. 이번 주에 할 일
3. 막히거나 도움이 필요한 점

매 스크럼마다 노션으로 문서화를 진행함

팀 컨벤션

- 일정관리 / 출석 규칙
- 커뮤니케이션 규칙
- 작업 관리 규칙

협업에 필요한 회의·소통·작업 약속들을 1주차에 정하고 시작해서 불필요한 충돌이나 커뮤니케이션 혼선을 줄일 수 있었습니다.

AI 활용 내역

AI 활용 - 활용 개요

주 도구: Claude Code Opus 4.8 / Codex gpt-5.5 High / OpenCode 1.2.15

디자인: Google Stitch

책임 있는 AI 사용

활용 원칙

: AI가 코드를 쓰되, 팀 컨벤션(CLAUDE.md)·문서(SoT)를 따르게 하고,

최종 결정·리뷰·커밋은 사람이 직접 수행

- AI는 자동 commit/push 금지 – 사람이 검토 후 직접 반영
- 민감정보(.env / API key) 출력/커밋 차단 규칙 적용
- AI 결과를 그대로 신뢰하지 않고 실행·수동QA·코드 리뷰로 검증
- 문서가 코드와 충돌하면 사람이 SoT 기준으로 정리

AI 활용 - 문서화

docs/ - 합의된 기준 (SoT)

PRD.md	제품 요구사항	DESIGN.md	디자인 시스템 - SCSS 토큰·컴포넌트 스펙
REQUIREMENT.md	요구사항 분석서 (MoSCoW·NFR·제약·리스트·추적 매트릭스)	ARCHITECTURE.md	시스템 구조 - 폴더·데이터·인증 흐름·Firebase 모델
FuncDescription.md	기능명세서 - F-ID 별 입력/처리/출력/예외/인수기준	CLAUDE.md	AI/팀 협업 가이드 - 스택·컨벤션·도메인 용어집·금지사항
IA.md	화면 트리·URL·사용자 동선·GNB/LNB	RESPONSIVE.md	반응형 대응 계획

AI 활용 - 기능 구현 (코드 작성)

/goal 기반 개발 (작업 범위 관리)

조건이 충족될 때까지 Claude가 작업을 이어가도록 active goal을 설정하고

/goal [condition | clear]

Prompt: OpenSpec Requirement.md와 FunctionalDescription.md를 읽고, [기능] 완성을 위해 Goal #개로 나눠 주세요. 각 Goal마다: - 관련 OpenSpec task - 관련 requirement - Done When - 구현에서 제외할 것 을 정리해 주세요.

/loop 기반 검증 (자율 개선)

/loop [interval] [prompt]

/loop OpenSpec tasks.md를 기준으로 아직 완료되지 않은 task를 확인하고, 다음으로 구현할 가장 작은 작업을 제안해. # 파일 수정 전 계획만

각 Goal 완료 후 /verify 명령을 사용하여

- 요구사항 충족 여부
- 기능 동작 여부
- 범위 초과 구현 여부

결론 및 회고

[민지] 결론 및 회고

Keep (잘한 점 · 유지할 것)

- **문서 기반 개발을 유지한 점:** 특히 기능을 구현하기 전 `REQ -> F-ID -> 화면 -> 코드`로 연결되는 기준을 둔 점이 좋았다. 구인, 신청, 알림, 회고록처럼 서로 연결되는 도메인에서도 "어떤 정책을 따라야 하는지"를 문서에서 확인할 수 있어 임의 구현을 줄일 수 있었다.
- **도메인 단위로 역할을 나눈 점:** 작업을 단순히 프론트엔드/백엔드처럼 가로로 나누지 않고, 구인, 신청, 참고 주제, 회고록, 알림, 프로필처럼 도메인 단위로 나누었다. 한 사람이 화면, 혹, API, 문서까지 세로로 책임지는 구조였기 때문에 기능별 완성도를 관리하기 쉬웠다.
- **Firestore Rules를 권한의 기준으로 둔 점:** 클라이언트에서 버튼을 숨기거나 접근을 막는 것에 그치지 않고, Firestore Security Rules를 실제 권한 검증의 기준으로 둔 점이 좋았다. 신청, 찜, 추천, 알림, 회고록, 첨부 접근처럼 사용자 권한이 중요한 기능에서 서버 측 규칙을 함께 고려했다.

Problem (아쉬운 점 · 문제)

- **초반 명칭과 스키마 기준이 흔들린 점:** 회고록은 코드에서 `Retrospective`, URL에서 `/memoirs`, DB 컬렉션에서 `memoirs`로 불렸고, 구인도 URL은 `/togethers`, 코드에서는 `Project` 계열 명칭을 사용했다. 의도적으로 분리한 부분도 있었지만, 초반에는 문서마다 명칭과 필드 기준이 달라 구현 직전 혼란이 있었다.
- **Firestore의 제약을 뒤늦게 체감한 점:** Firestore는 RDB처럼 자유롭게 `where`와 `orderBy`를 조합할 수 없고, 복합 인덱스가 없으면 쿼리가 실패한다. 회고록 목록, 찜 목록, 알림 목록에서 `where + orderBy` 조합 때문에 문제가 발생했다. 처음부터 Firestore 조회 패턴과 인덱스 전략을 더 명확히 설계했다면 시행착오를 줄일 수 있었다. 이후 단일 `where` 후 클라이언트 정렬이라는 컨벤션을 세웠지만, 이 기준이 프로젝트 초반부터 있었으면 더 좋았을 것이다.

Try (다음에 시도할 것)

- 실시간 알림 기능: 새로고침해야 알림이 업데이트되는게 아니라 다른 유저가 신청/승인/반려를 하면 실시간으로 알림이 업데이트 되도록 UX 개선 작업
- 포트폴리오 프레임워크 분석 기능 정밀화: 실제 내가 사용하는 프레임워크를 더 정확하게 추적하도록 가중치 및 설정 파일 검색 개선 작업

[다연] 결론 및 회고

Keep (잘한 점 · 유지할 것)

- **화면, 로직, Firebase 접근 계층을 분리한 점:** 프로젝트 구조를 `pages/components -> hooks -> api -> Firebase` 흐름으로 나눈 점은 유지할 만하다. 화면 컴포넌트가 Firestore를 직접 호출하지 않고, 훅이 상태와 비즈니스 흐름을 담당하며, API 파일이 Firebase 접근을 담당하도록 분리했다. 이 구조 덕분에 특정 기능에서 문제가 생겼을 때 UI 문제인지, 상태 관리 문제인지, Firestore 쿼리 문제인지 분리해서 확인할 수 있었다.

Problem (아쉬운 점 · 문제)

- **자동화 테스트가 부족한 점:** lint, build, Playwright 기반 화면 QA는 수행했지만, 핵심 도메인 로직에 대한 단위 테스트나 통합 테스트는 충분하지 않았다. 특히 신청 승인, 추천 수 트랜잭션, Rules 권한 검증, GitHub 포트폴리오 분석처럼 로직이 중요한 영역은 자동화 테스트가 있었다면 더욱 높은 퀄리티의 제품을 완성할 수 있었을 것 같다.

Try (다음에 시도할 것)

- 포트폴리오 PDF 자동 다운 기능 개발
- 숙명 학생들을 대상 타겟으로 잡은 것을 고려하여, 디자인에 숙명 메인 컬러와 상징 요소들 반영

참고 자료 (관련 링크)

참고자료

사이트 레퍼런스

- 디프만. (n.d.). 디프만 - Depromeet. 2026년 6월 18일 검색, <https://www.depromeet.com>
- 홀라. (n.d.). HOLA 홀라 | IT 사이드프로젝트·스터디 팀원 모집. 2026년 6월 18일 검색, <https://holaworld.io>

참고 기사

- 홍길동. (2025). 렛플, 사이드프로젝트2천개... 성공률은 5.9%. 플래툰(Platum). <https://platum.kr/archives/245593>
- 정단비. (2024). 사이드프로젝트시장의 현시점은?. 데일리팝. <https://www.dailypop.kr/news/articleView.html?idxno=80814>
- 렛플. (n.d.). 사이트 이용자 분석 통계 (Quest 323). <https://letspl.me/quest/323>
- 렛플. (n.d.). 사이트 이용자 분석 통계 (Quest 726). <https://letspl.me/quest/726>
- 렛플. (n.d.). 구글 애널리틱스 기반 사이드프로젝트통계 분석 (Quest 149). <https://letspl.me/quest/149>

AI

- GitHub. (2026). GitHub Copilot (Version 0.48.1) [Computer software]. <https://github.com/features/copilot>
- Anthropic. (2026). Claude (Version 3.5 Sonnet) [Large language model]. <https://www.anthropic.com/claude>
- AnomalyCo. (2026). OpenCode (Version 0.1.x) [Computer software]. <https://opencode.ai/>
- Google. (2026). Gemini (Version 3.5 Flash) [Large language model]. <https://gemini.google.com/>
- OpenAI. (2026). OpenAI Codex [Large language model]. <https://openai.com/blog/openai-codex/>

스터디

- 하네스 엔지니어링 스터디 https://glib-stem-982.notion.site/AI-STUDY-1-356fc14745848055b42de51699036edf?source=copy_link